

Entrega 2

TFM-EQUIPO ETHOS

Estudiantes:

Jannyn Mendoza

Juan Mateo Duque

Nicole Custode

María Isabel Sanguino

Sacha Aponte

INSTITUCIÓN: INESDI - Máster Business Analytics e IA

FECHA: 08 de octubre de 2026

OBJETIVOS DEL PROYECTO:

1. Segmentar clientes según el tipo de cuenta que posee y analizar el comportamiento financiero, identificar cual es el tipo de cuenta con mayor y menor cantidad de clientes y analizar el por qué.
2. Analizar el tipo de cliente según su perfil y necesidades e identificar la cantidad de clientes que poseen o no inversión
3. Crear 3 productos financieros de inversión segmentado en rangos de edades, que permita generar rentabilidad y liquidez para ser utilizado en un plazo determinado con un fin específico.

MODELO DE DATOS DEL DATAWAREHOUSE EN MySQL (base de datos bancosabadell_dw)

Tabla de hechos (prefijo fact_) · Tabla de dimensión (prefijo dim_)

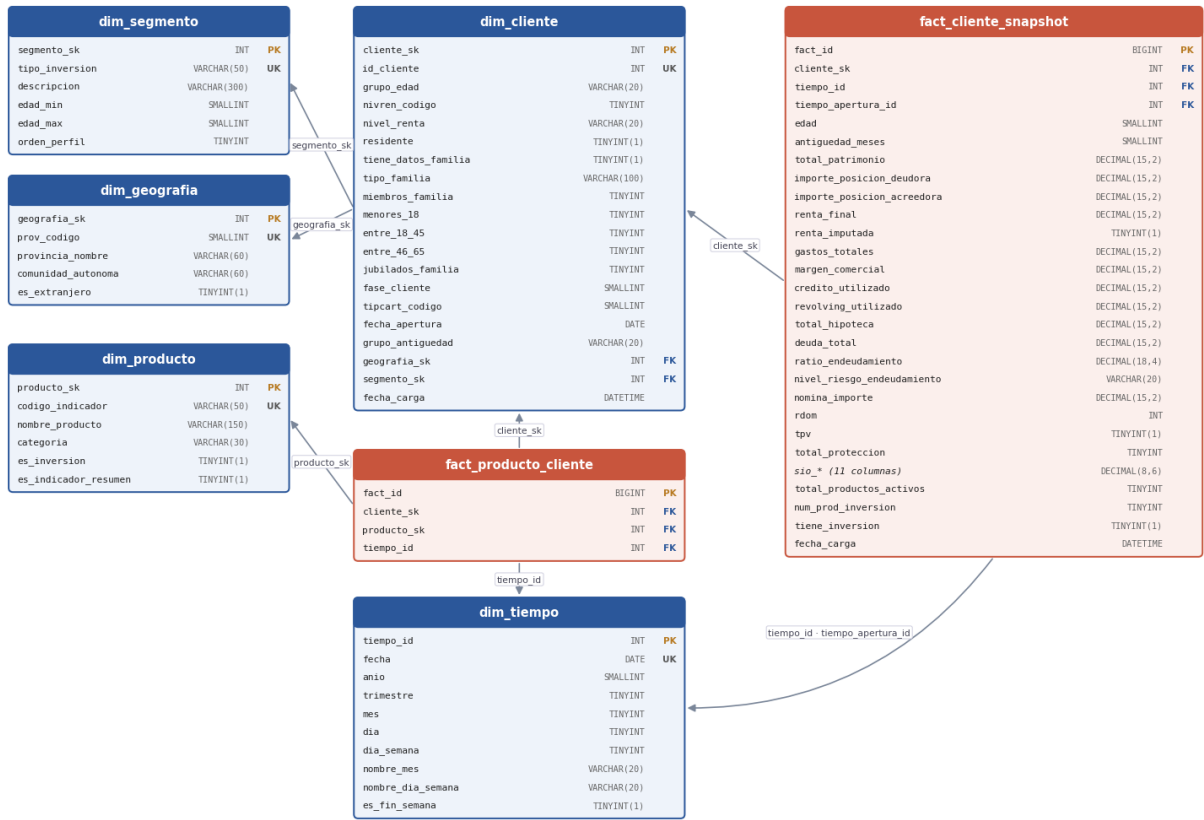
Tabla agregada (prefijo agg_) · Tabla de KPIs (prefijo kpi_)

PK = clave primaria ·

FK = clave foránea ·

UK = columna con restricción UNIQUE

Modelo del Datawarehouse — bancosabadell_dw (MySQL)



Tablas agregadas para la explotación (PK compuesta: tiempo_id + clave de agrupación; FK → dimensiones)



Tablas de KPIs (definición · ejecuciones · resultados históricos)

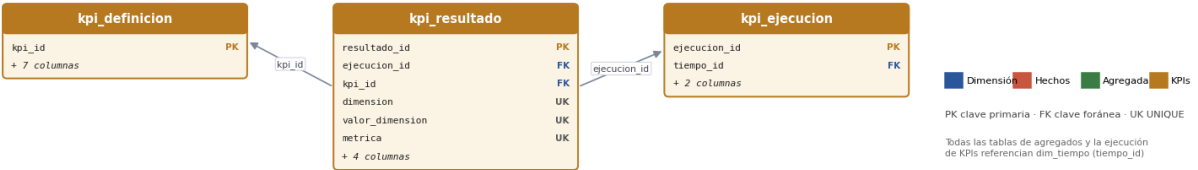


Figura 1. Modelo del Datawarehouse bancosabadell_dw: dimensiones, tablas de hechos, tablas agregadas y tablas de KPIs.

Tabla	Tipo	Clave primaria	Claves foráneas	Restricciones UNIQUE adicionales
dim_segmento	Dimensión	segmento_sk	—	tipo_inversion
dim_geografia	Dimensión	geografia_sk	—	prov_codigo
dim_producto	Dimensión	producto_sk	—	codigo_indicador
dim_tiempo	Dimensión	tiempo_id	—	fecha
dim_cliente	Dimensión	cliente_sk	geografia_sk → dim_geografia; segmento_sk → dim_segmento	id_cliente
fact_cliente_snapshot	Hechos	fact_id	cliente_sk → dim_cliente; tiempo_id → dim_tiempo; tiempo_apertura_id → dim_tiempo	uq_fact_cliente_snapshots (cliente_sk, tiempo_id)
fact_producto_cliente	Hechos	fact_id	cliente_sk → dim_cliente; producto_sk → dim_producto; tiempo_id → dim_tiempo	uq_fact_producto_cliente (cliente_sk, producto_sk, tiempo_id)
agg_segmento	Agregada	tiempo_id, segmento_sk	tiempo_id → dim_tiempo; segmento_sk → dim_segmento	—
agg_nivel_renta	Agregada	tiempo_id, nivren_codigo	tiempo_id → dim_tiempo	—
agg_provincia	Agregada	tiempo_id, geografia_sk	tiempo_id → dim_tiempo; geografia_sk → dim_geografia	—
agg_producto	Agregada	tiempo_id, producto_sk	tiempo_id → dim_tiempo; producto_sk → dim_producto	—
agg_distribucion_productos	Agregada	tiempo_id, numero_productos	tiempo_id → dim_tiempo	—
agg_top_clientes_patrimonio	Agregada	tiempo_id, ranking	tiempo_id → dim_tiempo; cliente_sk → dim_cliente	—
kpi_definicion	KPIs	kpi_id	—	—
kpi_ejecucion	KPIs	ejecucion_id	tiempo_id → dim_tiempo	—
kpi_resultado	KPIs	resultado_id	ejecucion_id → kpi_ejecucion; kpi_id → kpi_definicion	uq_kpi_resultado (ejecucion_id, kpi_id, dimension, valor_dimension, metrica)

Implementación: el modelo se crea desde el notebook (sección 5.2) en MySQL y se exporta al script modelo_dwh_mysql.sql. Los datos limpios se ingieren primero en la tabla stg_clientes_original, que crea el propio notebook, y a partir de ella Python construye las dimensiones, las tablas de hechos y las tablas agregadas.

Tablas de KPIs: kpi_definicion guarda el catálogo de los 7 KPIs (nombre, descripción, fórmula, unidad, frecuencia, objetivo del TFM y tabla origen); kpi_ejecucion registra cada ejecución del proceso (fecha, snapshot y nº de clientes) y kpi_resultado guarda los valores obtenidos en cada ejecución, lo que permite consultar su evolución histórica.

kpi_definicion

kpi_id	nombre	descripcion	formula	unidad	frecuencia	objetivo_tfm	tabla_origen
KPI_DISTRIBUCION_PRODUCTOS	Distribución de clientes por nº de productos	Clientes según cuántos productos tienen; identific...	numero_productos = SUM(45 ind_*); clientes, % y pa...	clientes / %	Mensual	Obj. 1 - vinculación y tipo de cuenta	agg_distribucion_productos
KPI_ENDEUDAMIENTO	Ratio de endeudamiento	Nivel de apalancamiento del cliente respecto a su ...	(credito_utilizado + revolving_utilizado + total_h...	ratio / %	Mensual	Obj. 2 - perfil financiero del cliente	fact_cliente_snapshot
KPI_PENETRACION_PRODUCTO	Penetración de producto	Grado de adopción de cada producto y categoría sob...	clientes_con_producto / total_clientes x 100, por ...	%	Mensual	Obj. 1 - tipo de cuenta con más y menos clientes	agg_producto
KPI_POR_NIVEL_RENTA	Cartera por nivel de renta	Concentración, rentabilidad y vinculación de la ca...	Por nivel_renta: COUNT clientes, AVG patrimonio, A...	clientes / € / nº	Mensual	Obj. 2 - perfil y necesidades del cliente	agg_nivel_renta
KPI_POR_PROVINCIA	Cartera por provincia	Distribución territorial de clientes, patrimonio y...	Por provincia: COUNT clientes, SUM y AVG patrimoni...	clientes / €	Mensual	Obj. 2 - segmentación de clientes	agg_provincia
KPI_POR_SEGMENTO	Comparativa de perfiles de inversión	Compara Juvenil, Conservador y Pensionado en tamañ...	Por tipo_inversion: COUNT, % clientes, AVG patrimo...	clientes / € / %	Mensual	Obj. 2 y 3 - productos de inversión por edad	agg_segmento
KPI_TOP_CLIENTES_PATRIMONIO	Concentración de patrimonio en el Top 100	Peso de los 100 clientes con más patrimonio sobre ...	SUM(patrimonio top 100) / SUM(patrimonio total) x ...	%	Mensual	Obj. 2 - clientes de mayor valor	agg_top_clientes_patrimonio

kpi_ejecucion

SELECT * FROM `kpi\_ejecucion`

☐ Perfilando

[\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

☐ Mostrar todo

Número de filas: 25

Filtrar filas:

Ordenar según la clave: Ninguna

Opciones extra

			ejecucion_id	fecha_ejecucion	tiempo_id	total_clientes
☐				682cea44-1d16-4c36-ae5b-2fe2ae573ab7	2026-09-30 04:54:21	20260911 9822
☐				b6b7267c-e93d-4e38-8731-7b707f992d2f	2026-09-29 18:57:01	20260911 9822
☐				be105012-faab-4f5b-a201-57465f8a7a3f	2026-09-29 20:05:29	20260911 9822

- kpi_resultado

resultado_id	ejecucion_id	kpi_id	dimension	valor_dimension	metrica	valor
3	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	0	total_clientes	1969.0000
4	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	0	pct_clientes	20.0500
5	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	0	patrimonio_medio	5141.8100
6	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	0	margen_medio	52.8000
7	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	1	total_clientes	2653.0000
8	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	1	pct_clientes	27.0100
9	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	1	patrimonio_medio	8026.6200
10	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	1	margen_medio	67.2300
11	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	2	total_clientes	1552.0000
12	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	2	pct_clientes	15.8000
13	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	2	patrimonio_medio	4725.2800
14	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	2	margen_medio	108.9100
15	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	3	total_clientes	1355.0000
16	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	3	pct_clientes	13.8000
17	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	3	patrimonio_medio	28556.2100
18	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	3	margen_medio	190.2500
19	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	4	total_clientes	1020.0000
20	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	4	pct_clientes	10.3800
21	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	4	patrimonio_medio	50722.1000
22	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	4	margen_medio	155.8200
23	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	5	total_clientes	688.0000
24	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	5	pct_clientes	7.0000
25	b6b7267c-e93d-4e38-8731-7b707f992d2f	KPI_DISTRIBUCION_PRODUCTOS	numero_productos	5	patrimonio_medio	51583.7100

KPI	Se calcula desde
KPI_DISTRIBUCION_PRODUCTOS	agg_distribucion_productos
KPI_POR_NIVEL_RENTA	agg_nivel_renta
KPI_POR_SEGMENTO	agg_segmento
KPI_PENETRACION_PRODUCTO	agg_producto
KPI_POR_PROVINCIA	agg_provincia
KPI_TOP_CLIENTES_PATRIMONIO	agg_top_clientes_patrimonio
KPI_ENDEUDAMIENTO	fact_cliente_snapshot (el ratio está calculado por cliente)

Todos los KPI se calculan en el notebook de Python leyendo las tablas del DWH en MySQL (hechos, dimensiones y agregados) y sus resultados se guardan en la tabla `kpi_resultado`, con una fila por KPI, dimensión y métrica en cada ejecución.

1. [KPI_DISTRIBUCION_PRODUCTOS](#)

Ficha del KPI	KPI_DISTRIBUCION_PRODUCTOS
Nombre	Distribución de clientes por nº de productos
Descripción	Clientes según cuántos productos tienen contratados; identifica clientes mono-producto (riesgo de fuga) frente a clientes muy vinculados.
Fórmula	$\text{numero_productos} = \text{suma de los 45 indicadores ind_* por cliente. Por cada valor: nº de clientes, \% de clientes, patrimonio medio y margen medio. KPI global: \% mono-producto} = \frac{\text{clientes con 1 producto}}{\text{total de clientes}} \times 100.$
Fuente (DWH)	<code>agg_distribucion_productos</code> (desde <code>fact_cliente_snapshot</code>)
Unidad	Clientes, % y €
Frecuencia	Mensual
Objetivo del TFM	Objetivo 1: vinculación y tipo de cuenta

Distribución de clientes por nº de productos contratados. Permite identificar cuántos clientes son **mono-producto** (riesgo de fuga) frente a los muy vinculados.

- `numero_productos` = suma de los 45 indicadores `ind_*` por cliente (`total_productos_activos` en el DWH).
- Para cada valor: total clientes, % clientes, `patrimonio_medio` y `margen_medio`.
- KPI global: **% de clientes mono-producto** y **% de clientes sin productos**.

Resultado (snapshot 11/09/2026):

- El **27,0 %** de los clientes es **mono-producto** y el **20,05 %** no tiene ningún producto activo: casi la mitad de la cartera tiene una vinculación débil.
- El valor del cliente crece con la vinculación: el patrimonio medio pasa de 5.142 € (0 productos) a 50.722 € (4 productos) y supera los 200.000 € a partir de 8 productos. El margen medio pasa de 53 € a 533 €.

Mostrando filas 0 - 11 (total de 12, La consulta tardó 0.0004 segundos.) [numero_productos: 0... - 11...]

```
SELECT * FROM `agg_distribucion_productos` ORDER BY `agg_distribucion_productos`.`numero_productos` ASC
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	tiempo_id	numero_productos	total_clientes	pct_clientes	patrimonio_medio	margen_medio
☐	20260911	0	1969	20.05	5141.81	52.80
☐	20260911	1	2663	27.01	8026.62	67.23
☐	20260911	2	1552	15.80	4725.28	108.91
☐	20260911	3	1355	13.80	28556.21	190.25
☐	20260911	4	1020	10.38	50722.10	155.82
☐	20260911	5	688	7.00	51593.71	177.38
☐	20260911	6	327	3.33	71101.42	401.63
☐	20260911	7	161	1.64	117074.02	461.34
☐	20260911	8	60	0.61	217806.75	533.35
☐	20260911	9	23	0.23	279030.35	738.12
☐	20260911	10	12	0.12	301837.01	720.63
☐	20260911	11	2	0.02	616645.87	988.38

Seleccionar todo | Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Figura 2. Tabla `agg_distribucion_productos` en MySQL (phpMyAdmin): clientes, % de clientes, patrimonio y margen medio por nº de productos.

2. KPI_ENDEUDAMIENTO

Ficha del KPI	KPI_ENDEUDAMIENTO
Nombre	Ratio de endeudamiento
Descripción	Nivel de apalancamiento del cliente respecto a su patrimonio en el banco; detecta clientes en riesgo de sobreendeudamiento.
Fórmula	ratio = (credito_utilizado + revolving_utilizado + total_hipoteca) / total_patrimonio. KPI global: % de clientes con ratio > 1.
Fuente (DWH)	fact_cliente_snapshot (ratio por cliente) + dim_segmento
Unidad	Ratio y %
Frecuencia	Mensual
Objetivo del TFM	Objetivo 2: perfil financiero del cliente

Permite medir el nivel de apalancamiento (deuda) de cada cliente en relación con su patrimonio, es decir, qué tan "endeudado" está cada uno respecto a lo que tiene.

$$\text{ratio_endeudamiento} = \frac{(\text{credito_utilizado} + \text{revolving_utilizado} + \text{total_hipoteca})}{\text{total_patrimonio}}$$

El ratio se calcula para cada cliente y se guarda en la tabla `fact_cliente_snapshot`, junto con `deuda_total` y `nivel_riesgo_endeudamiento` (Bajo $\leq 0,3$; Moderado $\leq 0,6$; Alto ≤ 1 ; Crítico > 1 ; Sin patrimonio). Como KPI se registra el % de clientes en riesgo crítico (ratio > 1), en total y por perfil de inversión, y la distribución de clientes por nivel de riesgo. Los umbrales son orientativos y pueden ajustarse.

Resultado (snapshot 11/09/2026):

- **1.425 clientes (14,5 %)** están en riesgo **crítico**: su deuda supera su patrimonio en el banco. Otros 347 clientes no tienen patrimonio y no se puede calcular su ratio.
- Por perfil: Pensionado 16,7 %, Conservador 8,3 %, Juvenil 0 %.
- El ratio mediano entre los clientes con deuda es 17,3, porque casi toda la deuda es hipotecaria.

¿Para qué sirve en la práctica?:

- Identificar clientes en riesgo financiero: una ratio alto (mucha deuda relativa a su patrimonio) es una señal de posible sobreendeudamiento — útil para priorizar seguimiento o gestión de riesgo de crédito.

orte_posicion_acreadora	renta_final	renta_imputada	gastos_totales	margen_comercial	credito_utilizado	revolving_utilizado	total_hipoteca	deuda_total	ratio_endeudamiento	nivel_riesgo_endeudamient
23100.89	16304.43	0	813.18	105.19	0.00	0.00	0.00	0.00	0.0000	Bajo
0.00	28711.49	1	0.00	32.42	0.00	0.00	0.00	0.00	0.0000	Bajo
400.20	11397.30	0	0.00	123.41	0.00	0.00	0.00	0.00	0.0000	Bajo
15317.10	18781.78	1	2881.17	22.51	0.00	0.00	0.00	0.00	0.0000	Bajo
0.00	19918.56	0	0.00	28.26	0.00	0.00	0.00	0.00	0.0000	Bajo
863.00	46992.93	0	316.79	352.87	0.00	0.00	0.00	0.00	0.0000	Bajo
0.00	16459.83	0	0.00	62.22	0.00	0.00	0.00	0.00	0.0000	Bajo
12165.53	61896.97	0	1278.64	116.15	0.00	0.00	0.00	0.00	0.0000	Bajo
650.00	16521.86	1	0.00	262.71	0.00	0.00	19508.18	19508.18	NULL	Sin patrimonio
1554.00	28740.91	0	522.88	-1.77	0.00	0.00	0.00	0.00	0.0000	Bajo
145772.84	176474.98	0	0.00	545.05	0.00	0.00	0.00	0.00	0.0000	Bajo
16012.41	21039.76	0	0.00	216.78	0.00	0.00	0.00	0.00	0.0000	Bajo
3016.94	22934.07	0	518.80	93.61	0.00	0.00	0.00	0.00	0.0000	Bajo
0.00	13410.99	0	0.00	4.67	0.00	0.00	0.00	0.00	0.0000	Bajo
31245.67	12850.72	0	549.67	26.47	0.00	0.00	0.00	0.00	0.0000	Bajo
8594.80	10696.87	0	0.00	5.19	0.00	0.00	0.00	0.00	0.0000	Bajo
7330.55	18933.79	0	0.00	1.59	0.00	0.00	0.00	0.00	0.0000	Bajo
18378.72	82246.56	0	2003.15	507.37	0.00	0.00	0.00	0.00	0.0000	Bajo
12212.51	17599.88	0	177.78	200.04	0.00	0.00	0.00	0.00	0.0000	Bajo
0.00	15056.60	0	0.00	73.80	0.00	0.00	0.00	0.00	0.0000	Bajo
118380.00	182473.36	0	0.00	20.63	0.00	0.00	0.00	0.00	0.0000	Bajo
21627.60	75431.40	0	677.07	88.95	0.00	0.00	0.00	0.00	0.0000	Bajo

Figura 3. Tabla `fact_cliente_snapshot` en MySQL: `deuda_total`, `ratio_endeudamiento` y `nivel_riesgo_endeudamiento` calculados por cliente.

3. KPI_POR_NIVEL_RENTA

Ficha del KPI	KPI_POR_NIVEL_RENTA
Nombre	Cartera por nivel de renta
Descripción	Concentración, rentabilidad, vinculación e inversión de la cartera según el nivel socioeconómico.
Fórmula	Por nivel de renta (nivren: Sin datos, Bajo ... Alto): nº y % de clientes, patrimonio medio, margen medio, productos medios y % de clientes con inversión.
Fuente (DWH)	agg_nivel_renta
Unidad	Clientes, %, € y nº de productos
Frecuencia	Mensual
Objetivo del TFM	Objetivo 2: perfil y necesidades del cliente

Este KPI es la base para priorizar en qué segmento de renta enfocar comercialmente — ya sea por volumen (Bajo/Medio-Bajo), por rentabilidad por cliente (Medio-Alto), o por oportunidad de cross-sell no explotado (Alto).

Permite identificar cómo se comporta la cartera de clientes según su nivel socioeconómico (renta), cruzando ese perfil con patrimonio, rentabilidad y vinculación. En concreto, sirve para analizar:

- Concentración de la cartera por poder adquisitivo
- Dónde está la rentabilidad real del banco.
- Nivel de vinculación (cross-sell) por poder adquisitivo.

Resultado (snapshot 11/09/2026):

- Volumen: los niveles Bajo y Medio-Bajo reúnen el **72,9 %** de los clientes (3.996 y 3.161 clientes).
- Rentabilidad: el nivel Medio-Alto (5,6 % de los clientes) tiene un margen medio de **321 €**, 3,5 productos por cliente y un 62,0 % de clientes con inversión.
- Oportunidad de venta cruzada: el nivel Alto tiene el mayor margen medio (**563 €**), pero menos productos (2,7) y menos inversión (41,5 %) que el Medio-Alto, por lo que tiene recorrido para productos de inversión.

Mostrando filas 0 - 5 (total de 6. La consulta tardó 0,0005 segundos.)

```
SELECT * FROM `agg_nivel_renta`
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	tiempo_id	nivren_codigo	nivel_renta	total_clientes	pct_clientes	patrimonio_medio	margen_medio	productos_medios	pct_con_inversion
☐ Editar ☐ Copiar ☐ Borrar	20200911	0	Sin datos	5	0.05	5636.57	27.10	0.80	20.00
☐ Editar ☐ Copiar ☐ Borrar	20200911	1	Bajo	3996	40.68	12228.41	73.84	1.76	22.82
☐ Editar ☐ Copiar ☐ Borrar	20200911	2	Medio-Bajo	3161	32.18	20754.22	134.68	2.13	35.68
☐ Editar ☐ Copiar ☐ Borrar	20200911	3	Medio	2067	21.04	25408.60	161.56	2.63	41.65
☐ Editar ☐ Copiar ☐ Borrar	20200911	4	Medio-Alto	552	5.62	101152.48	321.49	3.53	61.96
☐ Editar ☐ Copiar ☐ Borrar	20200911	5	Alto	41	0.42	200762.69	562.74	2.73	41.46

Seleccionar todo | Para los elementos que están marcados: ☐ Editar ☐ Copiar ☐ Borrar ☐ Exportar

Figura 4. Tabla `agg_nivel_renta` en MySQL: clientes, patrimonio, margen, productos e inversión por nivel de renta.

4. KPI_POR_SEGMENTO

Ficha del KPI	KPI_POR_SEGMENTO
Nombre	Comparativa de perfiles de inversión
Descripción	Compara los perfiles Juvenil, Conservador y Pensionado en tamaño, patrimonio, margen, vinculación e inversión.
Fórmula	Por tipo_inversion: nº y % de clientes, patrimonio medio, margen medio, % del margen total, productos medios, % con inversión y clientes sin inversión (mercado potencial).
Fuente (DWH)	agg_segmento + dim_segmento
Unidad	Clientes, % y €
Frecuencia	Mensual
Objetivo del TFM	Objetivos 2 y 3: productos de inversión por edad

KPI central para comparar los 3 perfiles de inversión (Juvenil, Conservador, Pensionado) en todas las dimensiones de gestión comercial. Es la base para diseñar los 3 productos de inversión del objetivo 3:

- **Juvenil:** inversión para estudios
- **Conservador:** inversión para la primera vivienda
- **Pensionado:** inversión para la jubilación

Resultado (snapshot 11/09/2026):

- **Pensionado:** 81,8 % de los clientes y **95,5 % del margen comercial**. Es el perfil a retener; aun así, 5.135 clientes todavía no invierten (mercado del producto de jubilación).
- **Conservador:** 10,3 % de los clientes, 4,0 % del margen y el porcentaje de inversión más bajo (16,0 %). Hay 852 clientes sin inversión para el producto de primera vivienda.
- **Juvenil:** 7,9 % de los clientes y solo el 0,4 % del margen (7 € de margen medio), pero con 2,6 productos por cliente, más vinculado que el Conservador. Hay 574 clientes sin inversión para el producto de estudios.

segmento_sk	total_clientes	pct_clientes	patrimonio_total	patrimonio_medio	margen_total	margen_medio	pct_margen	productos_medios	clientes_con_inversion	pct_con_inversion	r
1	774	7.88	1872760.31	2419.59	5465.53	7.06	0.44	2.64	200	25.84	
2	1014	10.32	4504394.18	4442.20	50573.89	49.88	4.03	1.76	162	15.98	
3	8034	81.80	224998884.08	27968.49	1199379.45	149.29	95.54	2.17	2899	36.08	

Figura 5. Tabla `agg_segmento` en MySQL: indicadores por perfil de inversión (Juvenil, Conservador, Pensionado).

5. KPI_PENETRACION_PRODUCTO

Ficha del KPI	KPI_PENETRACION_PRODUCTO
Nombre	Penetración de producto
Descripción	Grado de adopción de cada producto y de cada categoría sobre el total de clientes.
Fórmula	$pct_penetracion = clientes_con_producto / total_clientes \times 100$, por producto y por categoría (cada cliente cuenta una vez por categoría).
Fuente (DWH)	agg_producto + fact_producto_cliente + dim_producto
Unidad	%
Frecuencia	Mensual
Objetivo del TFM	Objetivo 1: tipo de cuenta con más y menos clientes

El objetivo de este KPI es medir el grado de adopción de cada uno de los 45 productos del banco sobre el total de la base de clientes, para identificar qué productos están fuertemente distribuidos y cuáles representan oportunidades de crecimiento comercial.

Cuantificar el alcance de cada producto — `clientes_con_producto`: cuántos clientes tienen contratado cada producto en el snapshot más reciente (se apoya en la tabla `fact_producto_cliente`). Normalizar ese alcance como tasa comparable — `pct_penetracion`: % de clientes con el producto sobre el total de la cartera ($clientes_con_producto / total_clientes \times 100$), lo que permite comparar productos entre sí sin sesgo por el tamaño total de la cartera.

Agrupar por categoría de negocio — `categoria` (Cuentas, Tarjetas, Inversion_Ahorro, Financiacion, Otros): permite analizar la penetración no solo producto a producto, sino a nivel de línea de negocio.

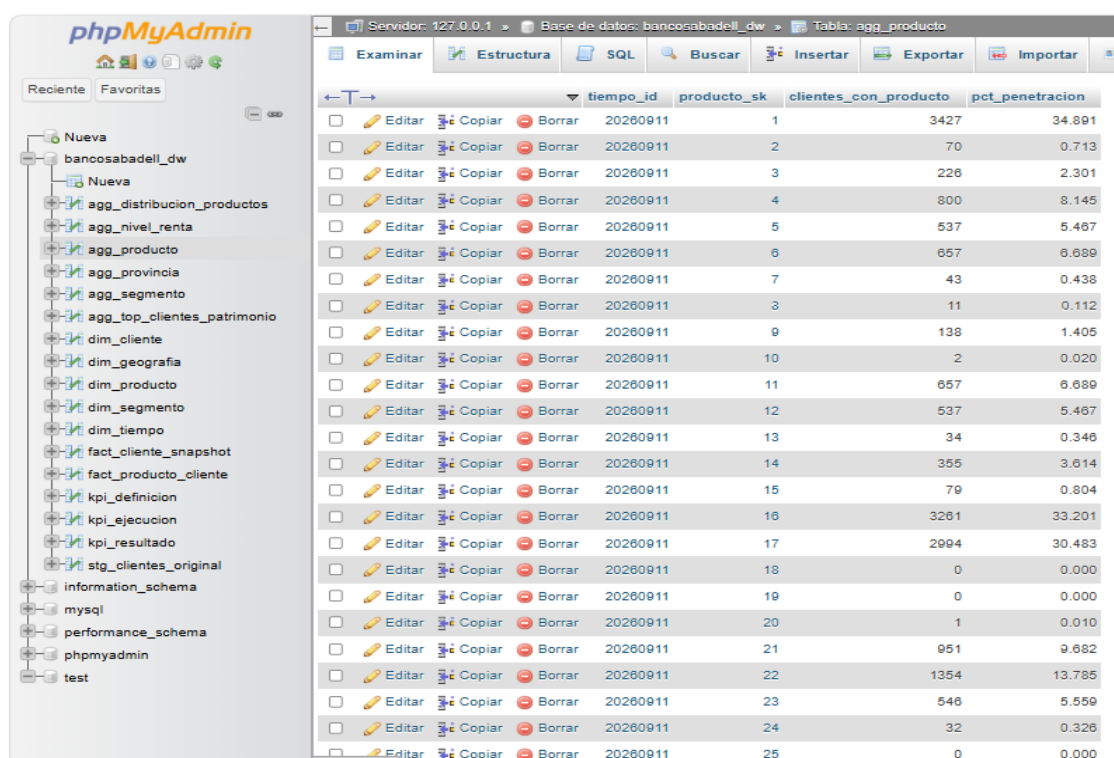
¿Para qué sirve en la práctica? (aplicaciones de negocio):

- Detectar productos con baja penetración, pero alto potencial → candidatos a campañas de venta cruzada (cross-selling).
- Identificar productos ya masificados (como la Cuenta Expansión, con un 34,9 % de penetración) que actúan como producto ancla de la relación con el cliente.

- Comparar categorías completas (p. ej. Tarjetas vs. Inversión/Ahorro) para orientar la estrategia comercial por la línea de producto.

Resultado (snapshot 11/09/2026):

- Por categoría: Cuentas 56,7 %, Inversión/Ahorro 33,2 %, Tarjetas 18,4 %, Financiación 14,7 %.
- El producto ancla es la **Cuenta Expansión** (34,9 %).
- **9 de los 45 productos no tienen ningún cliente** (por ejemplo, seguros de vida-ahorro colectivos y rentas temporales). Conviene tratarlos como productos inactivos en el análisis.



	tiempo_id	producto_sk	clientes_con_producto	pct_penetración
☐ Editar ☐ Copiar ☐ Borrar	20260911	1	3427	34.891
☐ Editar ☐ Copiar ☐ Borrar	20260911	2	70	0.713
☐ Editar ☐ Copiar ☐ Borrar	20260911	3	226	2.301
☐ Editar ☐ Copiar ☐ Borrar	20260911	4	800	8.145
☐ Editar ☐ Copiar ☐ Borrar	20260911	5	537	5.467
☐ Editar ☐ Copiar ☐ Borrar	20260911	6	657	6.689
☐ Editar ☐ Copiar ☐ Borrar	20260911	7	43	0.438
☐ Editar ☐ Copiar ☐ Borrar	20260911	3	11	0.112
☐ Editar ☐ Copiar ☐ Borrar	20260911	9	138	1.405
☐ Editar ☐ Copiar ☐ Borrar	20260911	10	2	0.020
☐ Editar ☐ Copiar ☐ Borrar	20260911	11	657	6.689
☐ Editar ☐ Copiar ☐ Borrar	20260911	12	537	5.467
☐ Editar ☐ Copiar ☐ Borrar	20260911	13	34	0.346
☐ Editar ☐ Copiar ☐ Borrar	20260911	14	355	3.614
☐ Editar ☐ Copiar ☐ Borrar	20260911	15	79	0.804
☐ Editar ☐ Copiar ☐ Borrar	20260911	16	3281	33.201
☐ Editar ☐ Copiar ☐ Borrar	20260911	17	2994	30.483
☐ Editar ☐ Copiar ☐ Borrar	20260911	18	0	0.000
☐ Editar ☐ Copiar ☐ Borrar	20260911	19	0	0.000
☐ Editar ☐ Copiar ☐ Borrar	20260911	20	1	0.010
☐ Editar ☐ Copiar ☐ Borrar	20260911	21	951	9.682
☐ Editar ☐ Copiar ☐ Borrar	20260911	22	1354	13.785
☐ Editar ☐ Copiar ☐ Borrar	20260911	23	546	5.559
☐ Editar ☐ Copiar ☐ Borrar	20260911	24	32	0.326
☐ Editar ☐ Copiar ☐ Borrar	20260911	25	0	0.000

Figura 6. Tabla agg_producto en MySQL: clientes con cada producto y % de penetración.

6. KPI_POR_PROVINCIA

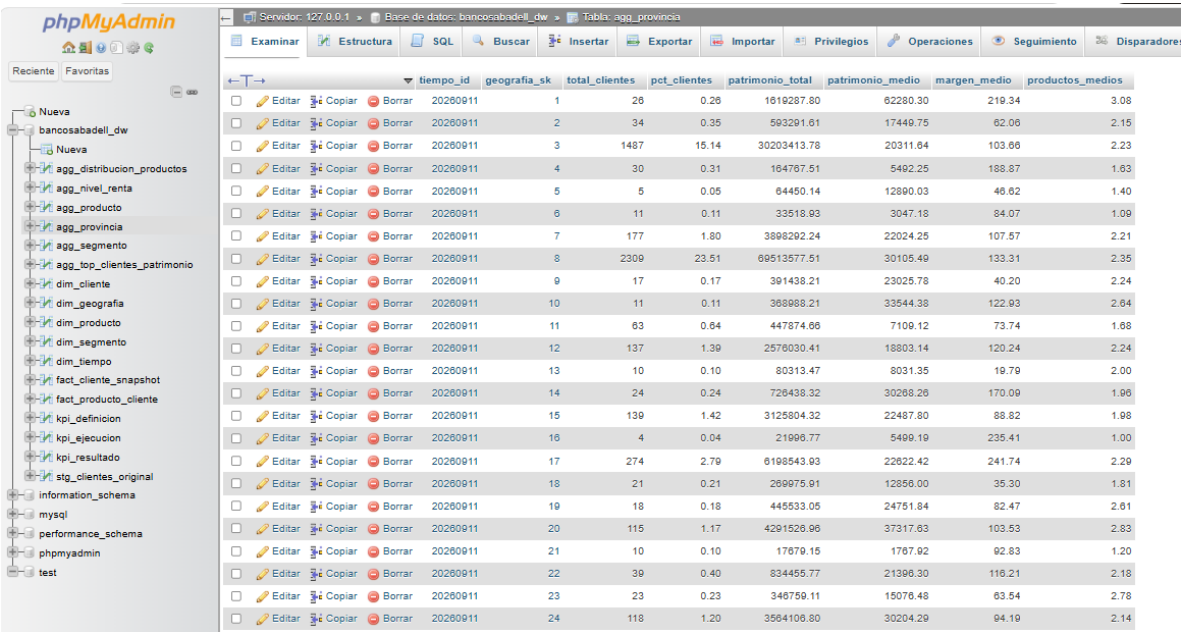
Ficha del KPI	KPI_POR_PROVINCIA
Nombre	Cartera por provincia
Descripción	Distribución territorial de clientes, patrimonio y margen.
Fórmula	Por provincia: nº y % de clientes, patrimonio total, patrimonio medio y margen medio. KPI global: % de clientes en las 3 primeras provincias.
Fuente (DWH)	agg_provincia + dim_geografia
Unidad	Clientes, % y €
Frecuencia	Mensual
Objetivo del TFM	Objetivo 2: segmentación de clientes

Segmenta y compara la cartera de clientes por provincia de residencia (código INE; 99 = residente en el extranjero) para identificar dónde se concentran los clientes, el patrimonio y el margen.

Ayuda a decidir **dónde lanzar primero** cada producto. Por ejemplo, el de jubilación en las provincias con mucho patrimonio medio y muchos Pensionados. **Decisiones de negocio:** dónde reforzar oficinas o gestores, dónde hacer campañas de captación (provincias con pocos clientes, pero de alto valor) y cuánto riesgo supone depender de pocas zonas.

Resultado (snapshot 11/09/2026):

- Barcelona (23,5 %), Alicante (15,1 %) y los residentes en el extranjero (8,1 %) reúnen el **46,7 %** de los clientes. Solo Barcelona concentra el 30,1 % del patrimonio total (69,5 M€ de 231,1 M€).
- Tener más clientes no implica tener clientes de más valor: Bizkaia (40.391 €) y Asturias (39.115 €) superan en patrimonio medio a Barcelona (30.105 €).
- Los 792 clientes residentes en el extranjero tienen poco patrimonio medio (12.664 €) pero un margen medio alto (148 €), por lo que forman un segmento propio.



tiempo_id	geografia_sk	total_clientes	pct_clientes	patrimonio_total	patrimonio_medio	margen_medio	productos_medios
20260911	1	26	0.28	1619287.80	62280.30	219.34	3.08
20260911	2	34	0.35	593291.61	17449.75	62.06	2.15
20260911	3	1487	15.14	30203413.78	20311.64	103.66	2.23
20260911	4	30	0.31	164707.51	5492.25	188.87	1.63
20260911	5	5	0.05	64450.14	12890.03	46.62	1.40
20260911	6	11	0.11	33518.93	3047.18	84.07	1.09
20260911	7	177	1.80	3898292.24	22024.25	107.57	2.21
20260911	8	2309	23.51	69513577.51	30105.49	133.31	2.35
20260911	9	17	0.17	391438.21	23025.78	40.20	2.24
20260911	10	11	0.11	368988.21	33544.38	122.93	2.64
20260911	11	63	0.64	447874.66	7109.12	73.74	1.88
20260911	12	137	1.39	2578030.41	18803.14	120.24	2.24
20260911	13	10	0.10	80313.47	8031.35	19.79	2.00
20260911	14	24	0.24	726438.32	30268.26	170.09	1.96
20260911	15	139	1.42	3125804.32	22487.80	88.82	1.98
20260911	16	4	0.04	21996.77	5499.19	235.41	1.00
20260911	17	274	2.79	6198543.93	22622.42	241.74	2.29
20260911	18	21	0.21	269975.91	12856.00	35.30	1.81
20260911	19	18	0.18	445533.05	24751.84	82.47	2.61
20260911	20	115	1.17	4291526.96	37317.63	103.53	2.83
20260911	21	10	0.10	17679.15	1767.92	92.83	1.20
20260911	22	39	0.40	834455.77	21396.30	116.21	2.18
20260911	23	23	0.23	346759.11	15078.48	63.54	2.78
20260911	24	118	1.20	3564106.80	30204.29	94.19	2.14

Figura 7. Tabla agg_provincia en MySQL: clientes, patrimonio y margen por provincia.

7. KPI_TOP_CLIENTES_PATRIMONIO

Ficha del KPI	KPI_TOP_CLIENTES_PATRIMONIO
Nombre	Concentración de patrimonio en el Top 100
Descripción	Peso de los 100 clientes con más patrimonio sobre el total de la cartera y detección de fuga de margen.
Fórmula	pct_patrimonio_top100 = suma del patrimonio de los 100 primeros / patrimonio total × 100. Además: nº de clientes del Top 100 con margen inferior a la mediana de la cartera.
Fuente (DWH)	agg_top_clientes_patrimonio + dim_cliente
Unidad	% y clientes
Frecuencia	Mensual
Objetivo del TFM	Objetivo 2: clientes de mayor valor

Identificar y listar a los 100 clientes con mayor patrimonio de toda la cartera, con su contexto de segmento, renta y ubicación, para su revisión manual por parte de gestores comerciales o de banca privada.

¿Para qué sirve en la práctica?:

- Priorizar la atención comercial sobre la cartera de mayor valor (clientes VIP / banca privada).
- Detectar **fuga de margen**: clientes con patrimonio alto pero margen_comercial bajo o negativo — son clientes valiosos que el banco está monetizando mal.
- Servir de lista de trabajo directa para campañas de fidelización o revisión de riesgo concentrado en pocos clientes.

Resultado (snapshot 11/09/2026):

- Los 100 primeros clientes (1 % de la cartera) concentran el **24,1 %** del patrimonio total: un riesgo de concentración relevante.
- Todos pertenecen al perfil Pensionado; el primero concentra por sí solo el 2,1 % del patrimonio (4,8 M€).
- Solo 3 tienen un margen comercial inferior a la mediana de la cartera (35,90 €): son los candidatos a revisar por fuga de margen.

phpMyAdmin

Reciente Favoritas

Nueva

bancosabadell_dw

Nueva

agg_distribucion_productos

agg_nivel_renta

agg_producto

agg_provincia

agg_segmento

agg_top_clientes_patrimonio

dim_cliente

dim_geografia

dim_producto

dim_segmento

dim_tiempo

fact_cliente_snapshot

fact_producto_cliente

kpi_definicion

kpi_ejecucion

kpi_resultado

stg_clientes_original

information_schema

mysql

performance_schema

phpmyadmin

test

Server: 127.0.0.1 » Base de datos: bancosabadell_dw » Tabla: agg_top_clientes_patrimonio

Examinar

Estructura

SQL

Buscar

Insertar

Exportar

Importar

Privilegios

Operaciones

Seguimiento

+ - T +

tiempo_id

ranking

cliente_sk

total_patrimonio

pct_patrimonio_cartera

margen_comercial

total_productos_activos

Editar

Copiar

Borrar

20260911

1

688

4798534.49

2.077

10434.39

3

Editar

Copiar

Borrar

20260911

2

112

2978202.22

1.289

5405.80

8

Editar

Copiar

Borrar

20260911

3

817

1808010.92

0.782

3492.44

9

Editar

Copiar

Borrar

20260911

4

207

1245550.52

0.539

2122.51

3

Editar

Copiar

Borrar

20260911

5

2158

1157299.42

0.501

1725.92

11

Editar

Copiar

Borrar

20260911

6

422

1148056.67

0.496

1962.81

10

Editar

Copiar

Borrar

20260911

7

32

1081485.57

0.468

2413.57

5

Editar

Copiar

Borrar

20260911

8

495

1014844.69

0.439

3704.11

7

Editar

Copiar

Borrar

20260911

9

3916

962799.80

0.430

1856.87

4

Editar

Copiar

Borrar

20260911

10

2950

952979.06

0.412

2331.09

6

Editar

Copiar

Borrar

20260911

11

858

896137.71

0.388

1801.92

9

Editar

Copiar

Borrar

20260911

12

1602

825151.65

0.357

120.95

6

Editar

Copiar

Borrar

20260911

13

410

778952.75

0.337

960.34

5

Editar

Copiar

Borrar

20260911

14

8321

771906.95

0.334

3089.84

1

Editar

Copiar

Borrar

20260911

15

771

764463.63

0.331

1967.46

8

Editar

Copiar

Borrar

20260911

16

7457

744764.70

0.322

1175.74

4

Editar

Copiar

Borrar

20260911

17

3009

734427.45

0.318

639.60

7

Editar

Copiar

Borrar

20260911

18

401

728406.94

0.315

1061.18

4

Editar

Copiar

Borrar

20260911

19

701

660865.58

0.286

1779.83

8

Editar

Copiar

Borrar

20260911

20

467

639791.80

0.277

1547.79

7

Editar

Copiar

Borrar

20260911

21

510

622691.68

0.270

897.33

8

Editar

Copiar

Borrar

20260911

22

7634

616898.27

0.267

895.79

6

Editar

Copiar

Borrar

20260911

23

2796

592786.71

0.257

1729.85

4

Editar

Copiar

Borrar

20260911

24

3736

582572.89

0.252

1343.98

8

Editar

Copiar

Borrar

20260911

25

423

568962.23

0.246

2331.31

7

Figura 8. Tabla `agg_top_clientes_patrimonio` en MySQL: ranking de los 100 clientes con mayor patrimonio.

Los KPIs se calculan **leyendo del DWH en MySQL** (tablas de hechos, dimensiones y agregados) y su resultado se guarda en tres tablas:

- **kpi_definicion:** catálogo de los KPIs (nombre, descripción, fórmula, unidad, frecuencia, objetivo del TFM y tabla origen). Clave primaria: kpi_id.
- **kpi_ejecucion:** una fila por cada ejecución del proceso (fecha, snapshot y nº de clientes). Clave primaria: ejecucion_id.
- **kpi_resultado:** valores obtenidos en cada ejecución, en formato largo (KPI, dimensión, valor de la dimensión, métrica y valor). Clave primaria: resultado_id; restricción UNIQUE sobre (ejecución, KPI, dimensión, valor de la dimensión, métrica).

Cada vez que se ejecuta el notebook se añade una nueva ejecución, de modo que kpi_resultado conserva el **histórico** de resultados. El valor global de cada KPI se guarda con dimension = 'TOTAL'.

Importación de librerías Para comenzar con la revisión de los datos, se procede a importar librerías especializadas de Python que permiten abordar cada una de las fases del análisis de datos: desde la manipulación y transformación de la información hasta la representación gráfica de los resultados. Pandas y NumPy proporcionan un entorno robusto para el procesamiento eficiente de registros, mientras que Matplotlib, Seaborn y Plotly permiten convertir grandes volúmenes de datos en representaciones visuales fáciles de interpretar. Esta combinación facilita detectar patrones, tendencias y anomalías que podrían pasar desapercibidas mediante una revisión únicamente tabular.Aunque esta etapa no genera resultados estadísticos, nos permite establecer la infraestructura analítica necesaria para desarrollar un estudio escalable y real.

```
In [118... # Librerías utilizadas en el notebook
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

pd.set_option('display.max_columns', 100)
pd.set_option('display.width', 200)
print("Librerías importadas OK ✅")
```

Librerías importadas OK ✅

Una vez preparado el entorno de trabajo, hacemos la carga de la base de datos correspondiente a la cartera de clientes del Banco Sabadell. El conjunto de datos representa el comportamiento financiero de aproximadamente 10.000 de clientes El tamaño de la muestra permite realizar segmentaciones por grupos de edad, patrimonio, antigüedad y contratación de productos sin perder significancia estadística, lo cual incrementa la confiabilidad de los resultados obtenidos posteriormente. Antes de realizar cualquier proceso de limpieza o modelado resulta imprescindible conocer la estructura de la información disponible. Para ello se investigaron las primeras filas del dataset y lacomposición de las variables. Este procedimiento permite comprender cómo se encuentra organizada la información y detectar posibles inconsistencias desde una perspectiva preliminar.

```
In [119... input_dir = Path.cwd() / "input"
xlsx_files = []
if input_dir.exists():
    xlsx_files = sorted(input_dir.glob("*.xlsx"))
else:
    xlsx_files = sorted(Path.cwd().glob("*.xlsx"))

if not xlsx_files:
    raise FileNotFoundError(
        f"No se encontró ningún archivo .xlsx en {input_dir} ni en el directorio actual."
    )

excel_path = xlsx_files[0]
df_datosabadell = pd.read_excel(excel_path, engine="openpyxl")
print(f"Listo✅! Excel cargado desde: {excel_path}")
print(f"output_dir será: {Path.cwd() / 'output'}")
```

Listo✅! Excel cargado desde: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 2\BD_Sabadell.xlsx
output_dir será: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 2\output

El siguiente análisis es para revisar medidas estadísticas descriptivas para así resumir miles de registros en indicadores fácilmente interpretables.

```
In [120... # Estadística descriptiva de las variables clave de perfil / consumo
vars_clave = ['antig','edad','totalpasta','renta_final','gastos_tot','nivren',
              'margecomercialut','credit','revolving','tothipo','nominainp','rdom','tpv']
df_datosabadell[vars_clave].describe().T
```

Out[120...

	count	mean	std	min	25%	50%	75%	max
antig	10000.0	218.333500	795.911927	1.00	47.000000	121.000	233.000000	9999.00
edad	10000.0	49.354400	20.248531	0.00	36.000000	49.000	64.000000	116.00
totalpasta	10000.0	23847.890669	86505.052000	0.00	287.467500	2123.225	17078.425000	4798534.49
renta_final	7239.0	25369.533403	81677.699690	4367.76	12310.628077	16479.540	25385.683636	4496606.59
nivren	10000.0	1.933000	0.937869	0.00	1.000000	2.000	3.000000	5.00
margecomercialut	10000.0	128.151498	462.907653	-1961.80	6.077500	36.050	138.195000	31639.73
credit	10000.0	100.536738	2457.438350	0.00	0.000000	0.000	0.000000	134200.66
revolving	10000.0	53.858220	332.113919	0.00	0.000000	0.000	0.000000	7901.99
tothipo	10000.0	13616.248837	52116.240914	0.00	0.000000	0.000	0.000000	2538920.97
nominainp	10000.0	1141.936248	4359.877628	0.00	0.000000	0.000	0.000000	335186.92
rdom	10000.0	8.978100	11.710924	0.00	0.000000	5.000	14.000000	86.00
tpv	10000.0	0.000700	0.026450	0.00	0.000000	0.000	0.000000	1.00

Se observó que algunas variables presentan una elevada dispersión, indicando que la cartera de clientes está compuesta por perfiles financieros muy heterogéneos. Esta diversidad evidencia la necesidad de implementar estrategias de segmentación comercial en lugar de campañas masivas dirigidas a todos los clientes por igual, ya que tenemos diferentes formas de actuar y de gestionar el dinero por parte de grupos de clientes y segmentos.

Cálculo del número total de clientes: Se contabilizó el número total de clientes presentes en la base de datos con el objetivo de establecer una referencia que permitiera verificar el impacto de las diferentes fases de limpieza y transformación. Este indicador constituye un punto de control dentro del proyecto. Conocer el tamaño inicial de la muestra facilita comprobar posteriormente que las operaciones de depuración únicamente eliminan registros realmente inconsistentes y no reducen innecesariamente la representatividad del conjunto de datos.

```
In [121... #Función para contar la cantidad total de clientes en la BD
total_clientes=df_datosabadell.id_cliente.count()
print(f"El total de clientes registrados en la BD es : {total_clientes}")
```

El total de clientes registrados en la BD es : 10000

Perfilado estadístico y análisis inicial de calidad Se realizó un perfilado estadístico de las variables para identificar rangos, valores mínimos y máximos, posibles inconsistencias y comportamientos poco habituales. Durante esta revisión se identificaron varios aspectos relevantes: • clientes con edad igual a cero, correspondientes a cuentas infantiles; • clientes sin productos contratados; • registros sin patrimonio asociado; • variables con valores nulos significativos. En lugar de asumir automáticamente que estos registros eran erróneos, se analizaron desde la perspectiva del negocio bancario. Este enfoque evita eliminar datos válidos y mejora considerablemente la calidad del análisis posterior.

```
In [122... # Análisis:
# Edad =0.000000 corresponde a padres que crean una cuenta de ahorro para sus hijos
# totalpasta =0.000000 corresponde a los cliente que no tiene fondos.
# impoped = 0.000000 corresponde a los clientes que no tuvieron egresos en su cuenta
```

```
resumen = df_datosabadell.describe()
print(resumen)
```

file:///C:/Users/maria/Downloads/Entrega2NotebookTFMcorregido.html

75%	0.216148	0.000000	0.780718	0.549935	4.000000	1.000000	2.000000	2.000000	1.000000	3.000000
1.000000	180.000000									
max	0.987529	1.000000	0.989921	0.827549	15.000000	7.000000	11.000000	10.000000	8.000000	5.000000
1.000000	9999.000000									

	prov	credit	revolving	tothipo	nominainp	rdom	tpv	margecomercialut	total_proteccio	fase
count	10000.000000	10000.000000	10000.000000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	9886.000000
mean	26.041600	100.536738	53.858220	1.361625e+04	1141.936248	8.978100	0.00070	128.151498	0.223400	19.240036
std	25.616645	2457.438350	332.113919	5.211624e+04	4359.877628	11.710924	0.02645	462.907653	0.591208	9.887232
min	1.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.00000	-1961.800000	0.000000	0.000000
25%	8.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.00000	6.077500	0.000000	11.000000
50%	20.000000	0.000000	0.000000	0.000000e+00	0.000000	5.000000	0.00000	36.050000	0.000000	22.000000
75%	33.000000	0.000000	0.000000	0.000000e+00	0.000000	14.000000	0.00000	138.195000	0.000000	28.000000
max	99.000000	134200.660000	7901.990000	2.538921e+06	335186.920000	86.000000	1.00000	31639.730000	6.000000	30.000000

In [123...

```
# Panorama general: tipos de dato y nulos
resumen = pd.DataFrame({
    'dtype': df_datosabadell.dtypes,
    'nulos': df_datosabadell.isna().sum(),
    '%_nulos': (df_datosabadell.isna().mean()*100).round(1)
}).sort_values('%_nulos', ascending=False)
resumen.head(20)
```

Out[123...

	dtype	nulos	%_nulos
jubilados	float64	4966	49.7
entre18_45	float64	4966	49.7
entre46_65	float64	4966	49.7
tipofamilia	str	4966	49.7
menores18	float64	4966	49.7
miembrosfamilia	float64	4966	49.7
renta_final	float64	2761	27.6
sio_mediospago	float64	114	1.1
sio_transaccionalidad	float64	114	1.1
sio_ahorro	float64	114	1.1
sio_accionabilidad	float64	114	1.1
sio_aldia	float64	114	1.1
sio_financiacion	float64	114	1.1
sio_desvinculacion	float64	114	1.1
sio_captacion	float64	114	1.1
sio_vinculacion	float64	114	1.1
sio_noaccionable	float64	114	1.1
fase	float64	114	1.1
sio_proteccion	float64	114	1.1
ind_cvprestige	int64	0	0.0

Del análisis de la base de datos se desprende que:

- **Variables demográficas (49,7 % nulos, 4.966 registros).** jubilados , entre18_45 , entre46_65 , tipofamilia , menores18 y miembrosfamilia comparten exactamente la misma cifra de omisiones. El patrón confirma que la ausencia se origina en la fuente primaria (falta sistemática de información familiar para casi la mitad de los clientes) y no son huecos aislados. No se imputan: se conserva el nulo y en el modelo del DWH se añade el indicador tiene_datos_familia para poder filtrar.
- **renta_final (27,6 % nulos).** Los nulos siguen un patrón real: los clientes sin renta_final son más jóvenes (41,9 vs 52,2 años) y menos vinculados (0,24 vs 0,42). La ausencia del dato es en sí misma informativa (probablemente clientes jóvenes o nuevos sin cruce fiscal). Por eso no se elimina la columna: **primero se crea el flag renta_imputada** para no perder esa señal y después se imputa la mediana por nivren (que solo correlaciona 0,09 con renta_final , así que no es redundante).
- **Campos sio_* y fase (1,1 % nulos, 114 filas).** Se eliminan esas 114 filas, porque no tienen ninguna puntuación SIO. fase se imputa con la mediana.

In [124...

```
# 1) Filas sin puntuación SIO: se eliminan (114 registros, 1,1 %)
cols_sio = [
    'sio_vinculacion', 'sio_desvinculacion', 'sio_ahorro', 'sio_aldia',
    'sio_captacion', 'sio_financiacion', 'sio_mediospago',
    'sio_transaccionalidad', 'sio_proteccion', 'sio_noaccionable', 'sio_accionabilidad'
]
filas_antes = len(df_datosabadell)
df_datosabadell = df_datosabadell.dropna(subset=cols_sio).copy()
print(f"Filas eliminadas por SIO nulo: {filas_antes - len(df_datosabadell)}")

# 2) 'fase': imputación por mediana
df_datosabadell['fase'] = df_datosabadell['fase'].fillna(df_datosabadell['fase'].median())

# 3) 'renta_final': primero se guarda el flag "sin dato" y después se imputa la mediana por nivren
df_datosabadell['renta_imputada'] = df_datosabadell['renta_final'].isna().astype(int)
df_datosabadell['renta_final'] = (
    df_datosabadell.groupby('nivren')['renta_final']
    .transform(lambda x: x.fillna(x.median()))
)
print(f"Clientes con renta imputada (flag renta_imputada = 1): {df_datosabadell['renta_imputada'].sum()}")
```

Filas eliminadas por SIO nulo: 114

Clientes con renta imputada (flag renta_imputada = 1): 2757

Se comprube a eliminación de los valores nulos y el tratamiento de los datos

In [125...

```
resumen = pd.DataFrame({
    'dtype': df_datosabadell.dtypes,
    'nulos': df_datosabadell.isna().sum(),
    '%_nulos': (df_datosabadell.isna().mean()*100).round(1)
}).sort_values('%_nulos', ascending=False)
resumen.head(20)
```

Out[125...

	dtype	nulos	%_nulos
miembrosfamilia	float64	4920	49.8
tipofamilia	str	4920	49.8
entre46_65	float64	4920	49.8
jubilados	float64	4920	49.8
menores18	float64	4920	49.8
entre18_45	float64	4920	49.8
impoped	float64	0	0.0
impopeh	float64	0	0.0
ind_cuentaexpansionpremium	int64	0	0.0
ind_cuentaexperiencia	int64	0	0.0
ind_cuentaprimera	int64	0	0.0
ind_cuentaproyeccion	int64	0	0.0
ind_cvautonomos	int64	0	0.0
ind_cvdirect	int64	0	0.0
ind_cuentaexpansion	int64	0	0.0
ind_cuentaexpansionnegocios	int64	0	0.0
id_cliente	int64	0	0.0
antig	int64	0	0.0
edad	int64	0	0.0
totalpasta	float64	0	0.0

In [126...

```
#Análisis de Los datos atípicos, conteno e identificación
cantidad_9999 = (df_datosabadell['antig'] == 9999).sum()
print(f"Cantidad de registros con antig == '9999, posibles datos atípicos': {cantidad_9999}")
```

Cantidad de registros con antig == '9999, posibles datos atípicos': 64

In [127...

```
#Se tomo la decisión de eliminar Los 64 registros con esos valores antig=9999, de acuerdo al análisis realizado eran clientes que no forman parte del Banco
df_delete = df_datosabadell[df_datosabadell['antig'] == 9999].copy()
print(f"CORRECCIÓN: Registros con antig == 9999 (para eliminar): {len(df_delete)}")
```

CORRECCIÓN: Registros con antig == 9999 (para eliminar): 64

In [128...

```
# Para eliminar realmente del DataFrame principal
df_datosabadell = df_datosabadell[df_datosabadell['antig'] != 9999]

print("\nEliminados los registros con valores 9999")
```

Eliminados los registros con valores 9999

In [129...

```
# Validamos La eliminación de Los datos atípicos: 10.000 - 114 (SIO nulos) - 64 (antig = 9999) = 9.822 clientes
total_clientes = df_datosabadell.id_cliente.count()
print(f"El total de clientes registrados en la BD es : {total_clientes}")
```

El total de clientes registrados en la BD es : 9822

Se construyen las variables que soportan los objetivos 2 y 3:

- **Grupos de productos:** cuentas (ind_cuenta*), tarjetas (ind_cv*), inversión/ahorro, financiación y otros. El catálogo completo de los 45 indicadores está en CATALOGO_PRODUCTOS (más abajo).
- **tipo_inversion** : perfil de inversión del cliente (Juvenil ≤ 18 años, Conservador 19–30 años, Pensionado > 30 años), construido a partir de la **edad**.
- **segmento_edad** : Joven / Adulto / Senior.
- **num_prod_inversion** : nº de productos de **inversión/ahorro** que tiene el cliente (solo la categoría Inversion_Ahorro, no cuentas, tarjetas ni préstamos). Se excluye ind_estalinv , que en la fuente es un indicador resumen (vale 1 si el cliente tiene cualquier producto de ahorro/inversión).
- **tiene_inversion** : 1 si el cliente tiene al menos un producto de inversión/ahorro.
- **total_productos_activos** : nº total de productos del cliente (los 45 indicadores).
- **gastos_tot** llega como texto con formato moneda (" € 1,278.64", " € - "); se convierte a numérico en la preparación para la ingesta en MySQL.

In [130...

```
# Función para calcular tipo de inversión según edad
def calcular_tipo_inversion(edad):
    """Calcular tipo de inversión según edad"""
    try:
        edad_val = float(edad)
        if edad_val <= 18:
            return 'Inversión Perfil Juvenil'
        elif edad_val <= 30:
            return 'Inversión Perfil Conservador'
        else:
            return 'Inversión Perfil Pensionado'
    except:
        return 'Inversión Perfil Conservador' # Valor por defecto

# Calcular tipo de inversión para todos los clientes
df_datosabadell['tipo_inversion'] = df_datosabadell['edad'].apply(calcular_tipo_inversion)

# Mostrar estadísticas de distribución
print("Distribución de tipos de inversión:")
distribucion = df_datosabadell['tipo_inversion'].value_counts()
for tipo, count in distribucion.items():
    porcentaje = (count / len(df_datosabadell)) * 100
    print(f" - {tipo}: {count} clientes ({porcentaje:.1f}%)")
```

Distribución de tipos de inversión:
- Inversión Perfil Pensionado: 8034 clientes (81.8%)
- Inversión Perfil Conservador: 1014 clientes (10.3%)
- Inversión Perfil Juvenil: 774 clientes (7.9%)

In [131...

```
nulos_restantes = df_datosabadell.isna().sum()
nulos_restantes = nulos_restantes[nulos_restantes > 0]
```

```
print(f"\nFilas finales: {len(df_datosabadell)} | Columnas finales: {df_datosabadell.shape[1]}")
if nulos_restantes.empty:
    print("✅ No quedan valores nulos en el dataset.")
else:
    print("⚠️ Columnas con nulos restantes:")
    print(nulos_restantes)
```

Filas finales: 9822 | Columnas finales: 85
⚠️ Columnas con nulos restantes:
miembrosfamilia 4890
menores18 4890
entre18_45 4890
entre46_65 4890
jubilados 4890
tipofamilia 4890
dtype: int64

In [132...

```
#Segmentacion de clientes por tipo de cuenta
# Lista de columnas a seleccionar
columnas_seleccion = [
    'id_cliente',
    'ind_cuentaexpansion',
    'ind_cuentaexpansionnegocios',
    'ind_cuentaexpansionpremium',
    'ind_cuentaexperiencia',
    'ind_cuentaprimera',
    'ind_cuentaproyeccion',
    'ind_cvautonomos',
    'ind_cvdirect',
    'ind_cvhabit',
    'ind_cvjove',
    'ind_cvjunior',
    'ind_cvmass',
    'ind_cvprestige',
    'ind_cvsenior',
    'ind_estalinv',
    'ind_estalvi',
]

# Seleccionar las columnas (syntax correcta)
df_tipocuenta = df_datosabadell[columnas_seleccion].copy()
```

In [133...

```
# Función para realizar group by por tipo de cuenta y contar cuántas personas tienen ese tipo de cuenta (sumar cuando el tipo de cuenta sea igual a 1)
def group_by_tipo_cuenta(df_datosabadell, account_columns):

    # Verificar que las columnas existan en el DataFrame
    missing_columns = [col for col in account_columns if col not in df_datosabadell.columns]
    if missing_columns:
        print(f"Advertencia: Las siguientes columnas no existen en el DataFrame: {missing_columns}")
        # Remover columnas faltantes de la lista
        account_columns = [col for col in account_columns if col not in missing_columns]

    # Crear un DataFrame para almacenar los resultados
    results = []

    for account_type in account_columns:
        # Contar cuántas personas tienen esta cuenta (valor = 1)
        count = df_datosabadell[df_datosabadell[account_type] == 1][account_type].count()

        # Calcular porcentaje del total
        total_clients = df_datosabadell['id_cliente'].nunique() # Usar nunique para contar clientes únicos
        percentage = (count / total_clients) * 100 if total_clients > 0 else 0

        results.append({
            'Tipo de Cuenta': account_type,
            'Cantidad': count,
            'Porcentaje': f"{percentage:.2f}%"
        })

    # Crear DataFrame de resultados
    results_df_datosabadell = pd.DataFrame(results)

    # Ordenar por cantidad de personas de mayor a menor
    results_df_datosabadell = results_df_datosabadell.sort_values(by='Cantidad', ascending=False)

    return results_df_datosabadell

columnas_cuenta = [
    'ind_cuentaexpansion',
    'ind_cuentaexpansionnegocios',
    'ind_cuentaexpansionpremium',
    'ind_cuentaexperiencia',
    'ind_cuentaprimera',
    'ind_cuentaproyeccion',
    'ind_cvautonomos',
    'ind_cvdirect',
    'ind_cvhabit',
    'ind_cvjove',
    'ind_cvjunior',
    'ind_cvmass',
    'ind_cvprestige',
    'ind_cvsenior',
    'ind_estalinv',
    'ind_estalvi',
]
```

In [134...

```
# Usar la función con df_datosabadell en lugar de df_tipocuenta
# IMPORTANTE: Primero el DataFrame, luego las columnas
resultados = group_by_tipo_cuenta(df_datosabadell, columnas_cuenta)
print(resultados)
```

	Tipo de Cuenta	Cantidad	Porcentaje
0	ind_cuentaexpansion	3427	34.89%
14	ind_estalinv	3261	33.20%
15	ind_estalvi	2994	30.48%
3	ind_cuentaexperiencia	800	8.14%
5	ind_cuentaproyeccion	657	6.69%
9	ind_cvjove	657	6.69%
10	ind_cvjunior	537	5.47%
4	ind_cuentaprimera	537	5.47%
12	ind_cvprestige	355	3.61%
2	ind_cuentaexpansionpremium	226	2.30%
7	ind_cvdirect	138	1.41%
13	ind_cvsenior	79	0.80%
1	ind_cuentaexpansionnegocios	70	0.71%
11	ind_cvmas	34	0.35%
6	ind_cvautonomos	11	0.11%
8	ind_cvhabit	2	0.02%

```
In [135... #Se calcula el promedio de antigüedad del cliente, coincide con el POWER BI
promedio_antiguedad = (df_datosabadel['antig'].mean())
print(f"El promedio de antigüedad de los clientes es: {promedio_antiguedad}")
```

El promedio de antigüedad de los clientes es: 154.31948686621868

```
In [136... #Se calcula el min de antigüedad del cliente coincide con el POWER BI
minimo_antiguedad = (df_datosabadel['antig'].min())
print(f"El Mínimo de antigüedad de la BD de clientes es de: {minimo_antiguedad}")
```

El Mínimo de antigüedad de la BD de clientes es de: 1

```
In [137... # Contar cuántos clientes tienen cada indicador activo
conteo_indicadores = {} # <- DEFINIR EL DICCIONARIO PRIMERO

for col in columnas_seleccion:
    if col != 'id_cliente' and col in df_tipocuenta.columns:
        # Asumiendo que Los indicadores son binarios (0/1 o True/False)
        conteo = df_tipocuenta[col].sum() if df_tipocuenta[col].dtype in [int, float, bool] else len(df_tipocuenta[df_tipocuenta[col] == True])
        conteo_indicadores[col] = conteo # <- Ahora sí está definido
print(conteo_indicadores) # Mostrar todos los conteos, no solo el último
```

{'ind_cuentaexpansion': np.int64(3427), 'ind_cuentaexpansionnegocios': np.int64(70), 'ind_cuentaexpansionpremium': np.int64(226), 'ind_cuentaexperiencia': np.int64(800), 'ind_cuentaprimera': np.int64(537), 'ind_cuentaproyeccion': np.int64(657), 'ind_cvautonomos': np.int64(11), 'ind_cvdirect': np.int64(138), 'ind_cvhabit': np.int64(2), 'ind_cvjove': np.int64(657), 'ind_cvjunior': np.int64(537), 'ind_cvmas': np.int64(34), 'ind_cvprestige': np.int64(355), 'ind_cvsenior': np.int64(79), 'ind_estalinv': np.int64(3261), 'ind_estalvi': np.int64(2994)}

De acuerdo al análisis se determina los 10 tipos de cuentas con mayor clientes son:

La Concentración Masiva: Existe una fuerte concentración de clientes en los primeros tres productos financieros:

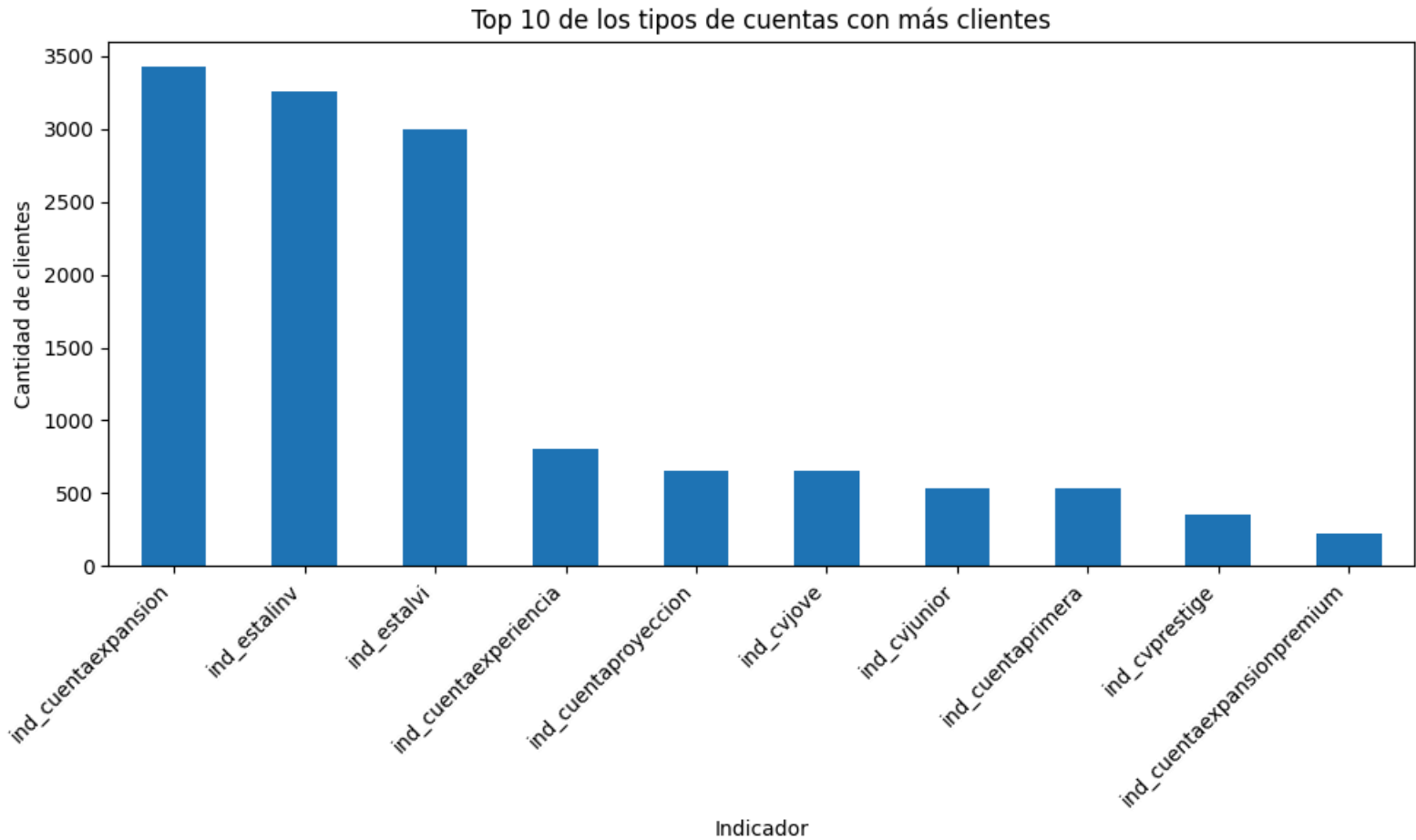
ind_cuentaexpansion: Es el tipo de cuenta líder, rozando los 3,500 clientes.

ind_estalinv: Ocupa el segundo lugar de forma muy cercana, superando los 3,300 clientes.

ind_estalvi: Ocupa el tercer puesto de la tipo de inversión principal con aproximadamente 3,000 clientes.

Caída Pronunciada (Efecto "Larga Cola"): A partir de la cuarta posición (ind_cuentaexperiencia), el volumen cae drásticamente por debajo de los 1,000 clientes (alrededor de 800), lo que representa una reducción de más del 70% en comparación con las cuentas top.

```
In [138... # Crear gráfico (funcionará ahora que conteo_indicadores está definido)
pd.Series(conteo_indicadores).sort_values(ascending=False).head(10).plot(
    kind='bar',
    title='Top 10 de los tipos de cuentas con más clientes',
    figsize=(10, 6)
)
plt.xlabel('Indicador')
plt.ylabel('Cantidad de clientes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Análisis: Relación entre Edad y Patrimonio Total Tendencia Positiva Moderada: Se observa una correlación positiva clara entre la edad y el patrimonio total. A mayor edad, la nube de puntos se desplaza hacia valores más altos de patrimonio (especialmente visible entre los 20 y los 60 años), lo que refleja el proceso de acumulación de ahorro a lo largo de la vida

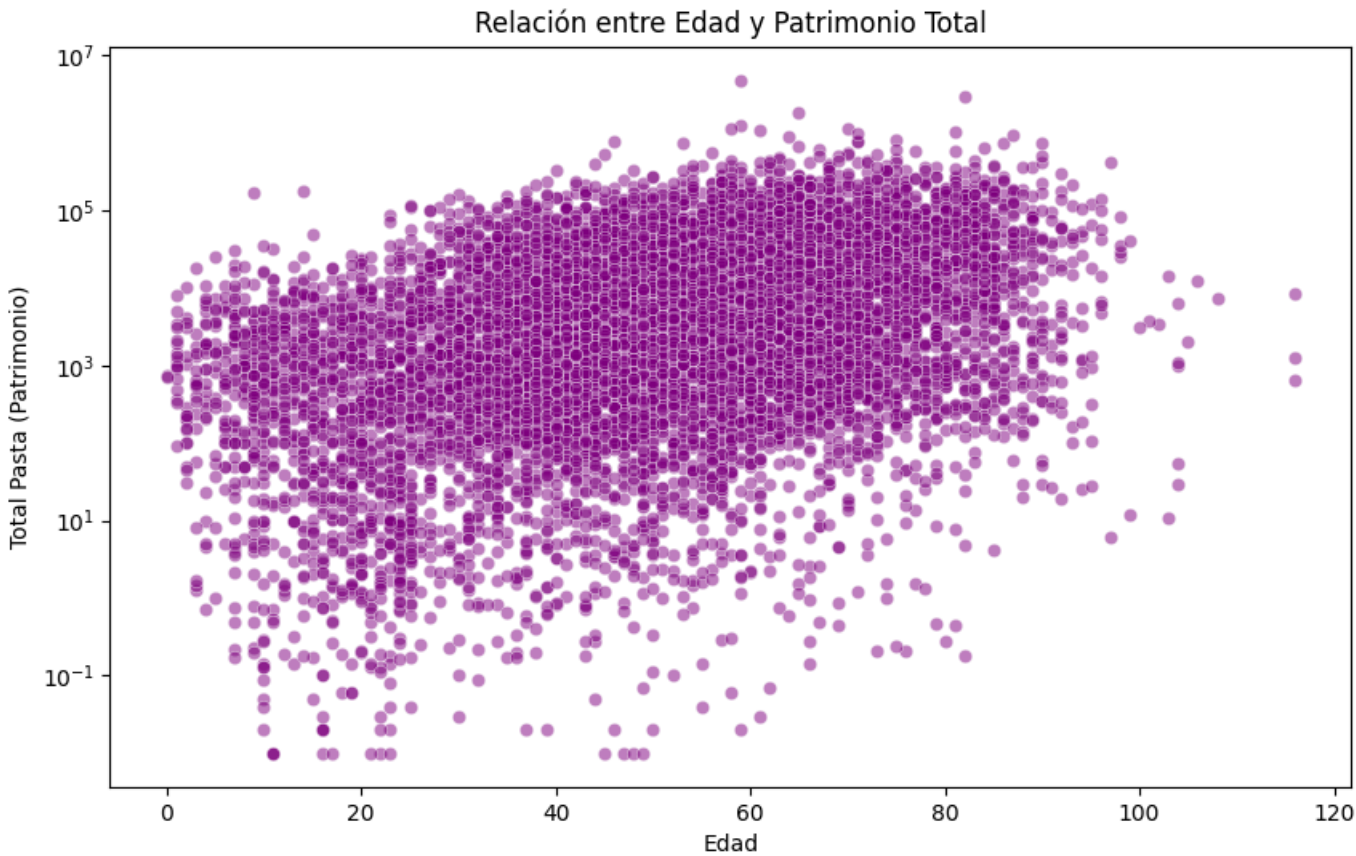
laboral.

Pico en la Madurez (40–70 años): La mayor densidad de clientes y los patrimonios más elevados (alcanzando escalas de 10^5 a 10^7) se concentran en el rango de 40 a 70 años.

Caída en Edades Avanzadas (>80 años): A partir de los 80 años se aprecia una disminución gradual en el número de clientes y en el volumen total del patrimonio, coherente con la etapa de jubilación y consumo de ahorros/herencias.

Alta Dispersión y Sesgo: A pesar de la escala logarítmica en el eje Y , existe una dispersión masiva en todos los rangos de edad (desde valores por debajo de 10^{-1} hasta más de 10^5), indicando que la edad por sí sola no determina completamente el nivel patrimonial, aunque sí marca su techo máximo.

```
In [139... plt.figure(figsize=(10, 6))
sns.scatterplot(x='edad', y='totalpasta', data=df_datosabadell, alpha=0.5, color='purple')
plt.title('Relación entre Edad y Patrimonio Total')
plt.xlabel('Edad')
plt.ylabel('Total Pasta (Patrimonio)')
plt.yscale('log') # Opcional si hay valores muy altos concentrados
plt.show()
```



En el gráfico que se muestra acontinuación se segmentan los clientes en 3 tipos de perfiles segun la edad -Inversión Perfil Pensionado con un total 8034 clientes corresponde al 81.8% del total de cliente, Inversión Perfil Conservador con 1014 clientes equivale al 10.3% e Inversión Perfil Juvenil con 774 clientes (7.9%)

```
In [140... # Gráfico de barras de La distribución de tipos de inversión
print("\n" + "="*50)
print("Gráfico de Barras: Distribución de Clientes por Tipo de Inversión")
print("="*50)

# Calcular la distribución
distribucion = df_datosabadell['tipo_inversion'].value_counts()

# Crear el gráfico de barras
plt.figure(figsize=(12, 7))

# Crear el gráfico de barras con colores según el tipo de inversión
colores = {
    'Inversión Perfil Juvenil': '#FF6B6B',    # Rojo/naranja para jóvenes
    'Inversión Perfil Conservador': '#4ECDC4', # Verde/azul para conservadores
    'Inversión Perfil Pensionado': '#45B7D1'   # Azul para pensionados
}

# Obtener colores en el orden correcto
bars = plt.bar(distribucion.index, distribucion.values,
               color=[colores[tipo] for tipo in distribucion.index],
               edgecolor='black', linewidth=1.5, alpha=0.8)

# Añadir etiquetas con los valores exactos
for i, bar in enumerate(bars):
    height = bar.get_height()
    porcentaje = (height / len(df_datosabadell)) * 100
    plt.text(bar.get_x() + bar.get_width()/2., height + 20,
             f'{int(height)} clientes\n({porcentaje:.1f}%)',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

# Personalizar el gráfico
plt.title('Distribución de Clientes por Tipo de Inversión', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Tipo de Inversión', fontsize=12, fontweight='bold')
plt.ylabel('Número de Clientes', fontsize=12, fontweight='bold')
plt.xticks(fontsize=11, rotation=0)
plt.yticks(fontsize=11)

# Añadir grid
plt.grid(axis='y', alpha=0.3, linestyle='--')

# Ajustar límites del eje Y
plt.ylim(0, max(distribucion.values) * 1.15)

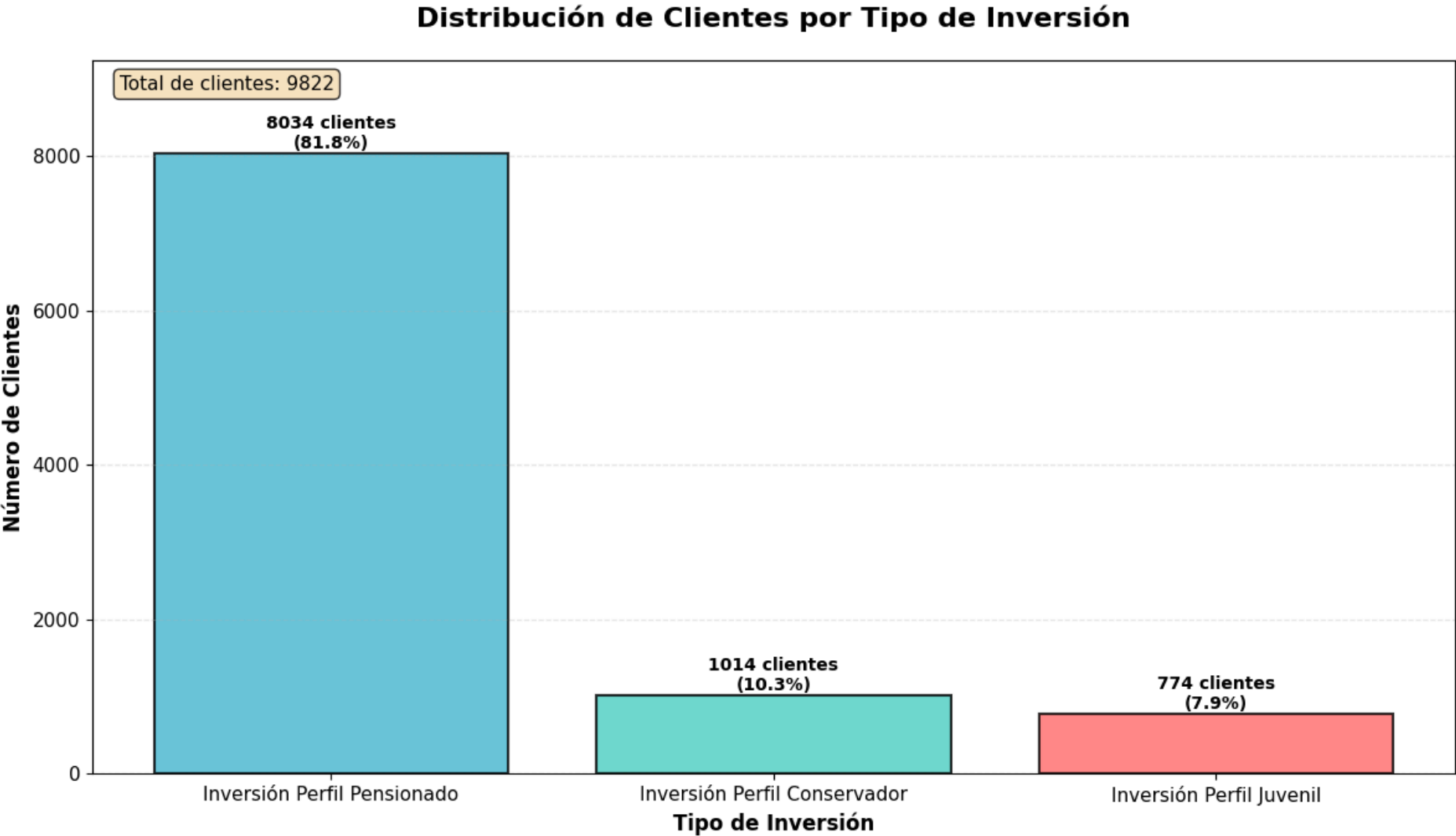
# Añadir anotación con el total de clientes
total_clientes = len(df_datosabadell)
plt.text(0.02, 0.98, f'Total de clientes: {total_clientes}',
        transform=plt.gca().transAxes, fontsize=11,
        verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

# Mostrar el gráfico
plt.tight_layout()
plt.show()
```

```
# Mostrar estadísticas adicionales
print("\nEstadísticas detalladas:")
for tipo, count in distribucion.items():
    porcentaje = (count / total_clientes) * 100
    print(f"• {tipo}: {count} clientes ({porcentaje:.1f}%)")

print(f"\nTotal general: {total_clientes} clientes")
print(f"Porcentaje total: {sum([(count / total_clientes) * 100 for count in distribucion.values]):.1f}%")
```

Gráfico de Barras: Distribución de Clientes por Tipo de Inversión



Estadísticas detalladas:

- Inversión Perfil Pensionado: 8034 clientes (81.8%)
- Inversión Perfil Conservador: 1014 clientes (10.3%)
- Inversión Perfil Juvenil: 774 clientes (7.9%)

Total general: 9822 clientes
Porcentaje total: 100.0%

Provincia del cliente. El dataset ya trae la provincia real en la columna `prov` (código INE; 99 = residente en el extranjero). A partir de ella se crean `provincia_nombre`, `comunidad_autonoma` y `es_extranjero`, que alimentan la dimensión `dim_geografia` del DWH.

En la versión anterior se generaba un `id_geografia` **aleatorio** (sin semilla, distinto en cada ejecución). Se elimina porque inventaba un dato que ya existía en la fuente.

```
In [141... # Códigos INE de provincia -> nombre y comunidad autónoma (99 = extranjero)
PROVINCIAS_INE = {
    1: "Araba/Álava", 2: "Albacete", 3: "Alicante/Alacant", 4: "Almería", 5: "Ávila",
    6: "Badajoz", 7: "Illes Balears", 8: "Barcelona", 9: "Burgos", 10: "Cáceres",
    11: "Cádiz", 12: "Castellón/Castelló", 13: "Ciudad Real", 14: "Córdoba", 15: "A Coruña",
    16: "Cuenca", 17: "Girona", 18: "Granada", 19: "Guadalajara", 20: "Gipuzkoa",
    21: "Huelva", 22: "Huesca", 23: "Jaén", 24: "León", 25: "Lleida",
    26: "La Rioja", 27: "Lugo", 28: "Madrid", 29: "Málaga", 30: "Murcia",
    31: "Navarra", 32: "Ourense", 33: "Asturias", 34: "Palencia", 35: "Las Palmas",
    36: "Pontevedra", 37: "Salamanca", 38: "Santa Cruz de Tenerife", 39: "Cantabria", 40: "Segovia",
    41: "Sevilla", 42: "Soria", 43: "Tarragona", 44: "Teruel", 45: "Toledo",
    46: "Valencia/València", 47: "Valladolid", 48: "Bizkaia", 49: "Zamora", 50: "Zaragoza",
    51: "Ceuta", 52: "Melilla", 99: "Extranjero",
}

CCAA_POR_PROVINCIA = {
    **dict.fromkeys([4, 11, 14, 18, 21, 23, 29, 41], "Andalucía"),
    **dict.fromkeys([22, 44, 50], "Aragón"),
    33: "Principado de Asturias", 7: "Illes Balears",
    **dict.fromkeys([35, 38], "Canarias"), 39: "Cantabria",
    **dict.fromkeys([5, 9, 24, 34, 37, 40, 42, 47, 49], "Castilla y León"),
    **dict.fromkeys([2, 13, 16, 19, 45], "Castilla-La Mancha"),
    **dict.fromkeys([8, 17, 25, 43], "Cataluña"),
    **dict.fromkeys([3, 12, 46], "Comunitat Valenciana"),
    **dict.fromkeys([6, 10], "Extremadura"),
    **dict.fromkeys([15, 27, 32, 36], "Galicia"),
    28: "Comunidad de Madrid", 30: "Región de Murcia", 31: "Navarra",
    **dict.fromkeys([1, 20, 48], "País Vasco"), 26: "La Rioja",
    51: "Ceuta", 52: "Melilla", 99: "Extranjero",
}

# Se elimina la columna aleatoria de la versión anterior, si existiera
df_datosabadell = df_datosabadell.drop(columns=['id_geografia'], errors='ignore')

df_datosabadell['provincia_nombre'] = df_datosabadell['prov'].map(PROVINCIAS_INE)
df_datosabadell['comunidad_autonoma'] = df_datosabadell['prov'].map(CCAA_POR_PROVINCIA)
df_datosabadell['es_extranjero'] = (df_datosabadell['prov'] == 99).astype(int)

sin_provincia = df_datosabadell['provincia_nombre'].isna().sum()
print(f"Provincias distintas en la cartera: {df_datosabadell['prov'].nunique()}")
print(f"Clientes con código de provincia no reconocido: {sin_provincia}")
```

```
print("\nTop 5 provincias por nº de clientes:")
print(df_datosabadell['provincia_nombre'].value_counts().head(5).to_string())
```

Provincias distintas en la cartera: 53
Clientes con código de provincia no reconocido: 0

Top 5 provincias por nº de clientes:

provincia_nombre	
Barcelona	2309
Alicante/Alacant	1487
Extranjero	792
Murcia	746
Madrid	724

Fecha de apertura de la cuenta. Se deriva de la antigüedad real del cliente (`antig` , en **meses**) respecto a la fecha de extracción de la cartera (`FECHA_SNAPSHOT`):

```
fecha_apertura = primer día del mes (FECHA_SNAPSHOT - antig meses)
```

Con ello se crean también `antigüedad_anios` y `grupo_antigüedad` (tramos en años). Como control de calidad, se comprueba que ningún cliente tenga una cuenta más antigua que su propia edad.

En la versión anterior `fecha_apertura` se generaba **al azar** dentro de los últimos 1.000 días, lo que contradecía `antig` (hasta 878 meses). Se sustituye por este cálculo determinista. *Supuesto a validar por el equipo:* `antig` está expresada en meses (mediana 120 = 10 años; máximo 878 ≈ 73 años, coherente con clientes de hasta 116 años).

In [142...

```
# Fecha de extracción de La cartera: parámetro del proceso ETL (se reutiliza como fecha del snapshot en el DWH)
FECHA_SNAPSHOT = pd.Timestamp('2026-09-11')

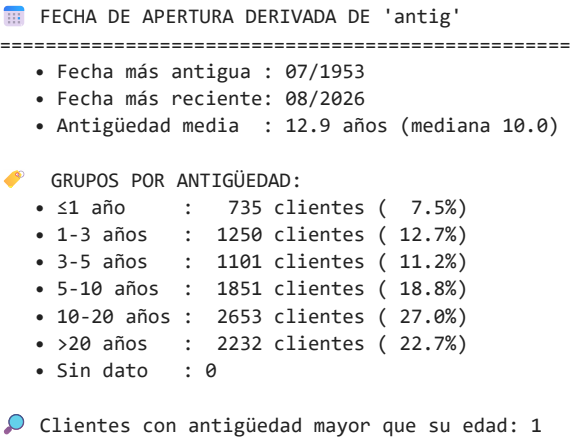
def calcular_fecha_apertura(df, fecha_ref=FECHA_SNAPSHOT, col_antig='antig'):
    """Deriva fecha_apertura (día 1 del mes) restando `antig` meses a la fecha de referencia."""
    out = df.copy()
    meses_totales = fecha_ref.year * 12 + (fecha_ref.month - 1) - out[col_antig].astype(int)
    out['fecha_apertura'] = pd.to_datetime(
        pd.DataFrame({'year': meses_totales // 12, 'month': meses_totales % 12 + 1, 'day': 1})
    )
    out['antigüedad_meses'] = out[col_antig].astype(int)
    out['antigüedad_anios'] = (out[col_antig] / 12).round(1)
    out['grupo_antigüedad'] = pd.cut(
        out[col_antig],
        bins=[0, 12, 36, 60, 120, 240, np.inf],
        labels=['≤1 año', '1-3 años', '3-5 años', '5-10 años', '10-20 años', '>20 años'],
        include_lowest=True,
    ).cat.add_categories('Sin dato').fillna('Sin dato').astype(str)
    return out

df_datosabadell = calcular_fecha_apertura(df_datosabadell)

print("\n 📊 FECHA DE APERTURA DERIVADA DE 'antig'")
print("=" * 50)
print(f"    • Fecha más antigua : {df_datosabadell['fecha_apertura'].min():%m/%Y}")
print(f"    • Fecha más reciente: {df_datosabadell['fecha_apertura'].max():%m/%Y}")
print(f"    • Antigüedad media  : {df_datosabadell['antigüedad_anios'].mean():.1f} años "
      f"(mediana {df_datosabadell['antigüedad_anios'].median():.1f})")

print("\n 📊 GRUPOS POR ANTIGÜEDAD:")
orden = ['≤1 año', '1-3 años', '3-5 años', '5-10 años', '10-20 años', '>20 años']
conteo = df_datosabadell['grupo_antigüedad'].value_counts().reindex(orden, fill_value=0)
for grupo, n in conteo.items():
    print(f"    • {grupo:<11}: {n:5} clientes ({n / len(df_datosabadell) * 100:5.1f}%)")
print(f"    • Sin dato    : {(df_datosabadell['grupo_antigüedad'] == 'Sin dato').sum()}")

# Control de calidad: La cuenta no puede ser más antigua que el cliente (margen de 1 año)
incoherentes = (df_datosabadell['antigüedad_anios'] > df_datosabadell['edad'] + 1).sum()
print(f"\n 🚩 Clientes con antigüedad mayor que su edad: {incoherentes}")
```



In [143...

```
# Catálogo de Los 45 indicadores de producto: código -> (nombre, categoría)
# Es La única definición del catálogo en el notebook: La reutilizan el DWH (dim_producto) y Los KPIs.
CATALOGO_PRODUCTOS = {
    # Cuentas
    'ind_cuentaexpansion': ('Cuenta Expansión', 'Cuentas'),
    'ind_cuentaexpansionnegocios': ('Cuenta Expansión Negocios', 'Cuentas'),
    'ind_cuentaexpansionpremium': ('Cuenta Expansión Premium', 'Cuentas'),
    'ind_cuentaexperiencia': ('Cuenta Experiencia', 'Cuentas'),
    'ind_cuentaprimera': ('Cuenta Primera', 'Cuentas'),
    'ind_cuentaproyeccion': ('Cuenta Proyección', 'Cuentas'),
    'ind_altrescc': ('Otras cuentas corrientes', 'Cuentas'),
    # Tarjetas
    'ind_cvautonomos': ('Tarjeta CV Autónomos', 'Tarjetas'),
    'ind_cvdirect': ('Tarjeta CV Direct', 'Tarjetas'),
    'ind_cvhabit': ('Tarjeta CV Habit', 'Tarjetas'),
    'ind_cvjove': ('Tarjeta CV Jove', 'Tarjetas'),
    'ind_cvjunior': ('Tarjeta CV Junior', 'Tarjetas'),
    'ind_cvmas': ('Tarjeta CV Más', 'Tarjetas'),
    'ind_cvprestige': ('Tarjeta CV Prestige', 'Tarjetas'),
    'ind_cvsenior': ('Tarjeta CV Senior', 'Tarjetas'),
    # Inversión y ahorro
    'ind_estalinv': ('Estalvi Inversión', 'Inversion_Ahorro'),
    'ind_estalvi': ('Estalvi (Ahorro)', 'Inversion_Ahorro'),
    'ind_asvidaestgarcol': ('Seguro Vida Ahorro Garantizado Colectivo', 'Inversion_Ahorro'),
```

```
'ind_asvidaestulinkc': ('Seguro Vida Ahorro Unit Linked Colectivo', 'Inversion_Ahorro'),
'ind_bonsestruc': ('Bonos estructurados', 'Inversion_Ahorro'),
'ind_cialp': ('CIALP (Cuenta Individual Ahorro L/P)', 'Inversion_Ahorro'),
'ind_diposit': ('Depósitos', 'Inversion_Ahorro'),
'ind_fi': ('Fondos de inversión', 'Inversion_Ahorro'),
'ind_fialtent': ('Fondos de inversión alternativos', 'Inversion_Ahorro'),
'ind_gcfsbp': ('Gestión de carteras', 'Inversion_Ahorro'),
'ind_gransel': ('Gran Selección', 'Inversion_Ahorro'),
'ind_pp': ('Planes de pensiones', 'Inversion_Ahorro'),
'ind_ppassoci': ('Planes de pensiones asociados', 'Inversion_Ahorro'),
'ind_ppemp': ('Planes de pensiones de empleo', 'Inversion_Ahorro'),
'ind_pprest': ('Planes de previsión (prestaciones)', 'Inversion_Ahorro'),
'ind_rentvitindiv': ('Renta vitalicia individual', 'Inversion_Ahorro'),
'ind_renttempcol': ('Renta temporal colectiva', 'Inversion_Ahorro'),
'ind_renttempind': ('Renta temporal individual', 'Inversion_Ahorro'),
'ind_restorvrf': ('Resto renta variable / renta fija', 'Inversion_Ahorro'),
'ind_rfixaval': ('Renta fija', 'Inversion_Ahorro'),
'ind_vrdafix': ('Valores de renta fija', 'Inversion_Ahorro'),
# Financiación
'ind_credf': ('Crédito', 'Financiacion'),
'ind_credv': ('Crédito vivienda', 'Financiacion'),
'ind_hipfix': ('Hipoteca tipo fijo', 'Financiacion'),
'ind_hipmixt': ('Hipoteca tipo mixto', 'Financiacion'),
'ind_hipoteca': ('Hipoteca', 'Financiacion'),
'ind_hipvar': ('Hipoteca tipo variable', 'Financiacion'),
'ind_promotor': ('Préstamo promotor', 'Financiacion'),
'ind_rentvehic': ('Renting de vehículo', 'Financiacion'),
# Otros
'ind_varispassiu': ('Otros productos de pasivo', 'Otros'),
}
IND_COLS = list(CATALOGO_PRODUCTOS)

# 'ind_estalinv' es un indicador RESUMEN de la fuente: vale 1 exactamente cuando el cliente tiene algún
# otro producto de ahorro/inversión. Se excluye del conteo para no contar dos veces el mismo producto.
INDICADORES_RESUMEN = ['ind_estalinv']
PRODUCTOS_INVERSION = [c for c, (_, cat) in CATALOGO_PRODUCTOS.items()
                        if cat == 'Inversion_Ahorro' and c not in INDICADORES_RESUMEN]

faltan = [c for c in IND_COLS if c not in df_datosabadell.columns]
assert not faltan, f"Faltan indicadores en el dataset: {faltan}"
print(f"Indicadores de producto: {len(IND_COLS)} | de inversión/ahorro: {len(PRODUCTOS_INVERSION)}")

def calcular_segmento_edad(edad):
    """Segmento de edad en 3 categorías: Joven (≤18), Adulto (19-45), Senior (>45)."""
    if pd.isna(edad):
        return 'Desconocido'
    if edad <= 18:
        return 'Joven'
    if edad <= 45:
        return 'Adulto'
    return 'Senior'

df_datosabadell['segmento_edad'] = df_datosabadell['edad'].apply(calcular_segmento_edad)
df_datosabadell['total_productos_activos'] = df_datosabadell[IND_COLS].sum(axis=1).astype(int)
df_datosabadell['num_prod_inversion'] = df_datosabadell[PRODUCTOS_INVERSION].sum(axis=1).astype(int)
df_datosabadell['tiene_inversion'] = (df_datosabadell['num_prod_inversion'] > 0).astype(int)

# Control de coherencia: tiene_inversion debe coincidir con el indicador resumen de la fuente
assert (df_datosabadell['tiene_inversion'] == df_datosabadell['ind_estalinv']).all(), \
    "tiene_inversion no coincide con ind_estalinv"
print("✓ tiene_inversion coincide al 100 % con el indicador resumen ind_estalinv de la fuente")

# Guardar el dataset actualizado (se entrega junto al notebook)
output_excel_path = Path.cwd() / "datosabadell_actualizado_con_segmentos.xlsx"
df_datosabadell.to_excel(output_excel_path, index=False)
print(f"\n✅ Dataset actualizado guardado en: {output_excel_path}")
print(f"    - Registros: {df_datosabadell.shape[0]} | Columnas: {df_datosabadell.shape[1]}")

print("\n📊 RESUMEN DE LAS NUEVAS COLUMNAS")
print(f"    • total_productos_activos: media {df_datosabadell['total_productos_activos'].mean():.2f} "
      f"    (mín {df_datosabadell['total_productos_activos'].min()}, máx {df_datosabadell['total_productos_activos'].max()})")
print(f"    • num_prod_inversion      : media {df_datosabadell['num_prod_inversion'].mean():.2f} "
      f"    (máx {df_datosabadell['num_prod_inversion'].max()})")
con_inv = df_datosabadell['tiene_inversion'].sum()
print(f"    • tiene_inversion          : {con_inv} clientes CON inversión ({con_inv / len(df_datosabadell) * 100:.1f}%), "
      f"    {len(df_datosabadell) - con_inv} SIN inversión")
print("\n    • segmento_edad:")
print(df_datosabadell['segmento_edad'].value_counts().to_string())
```

Indicadores de producto: 45 | de inversión/ahorro: 20
✓ tiene_inversion coincide al 100 % con el indicador resumen ind_estalinv de la fuente

✅ Dataset actualizado guardado en: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 2\datosabadell_actualizado_con_segmentos.xlsx
- Registros: 9822 | Columnas: 96

📊 RESUMEN DE LAS NUEVAS COLUMNAS

- total_productos_activos: media 2.16 (mín 0, máx 11)
- num_prod_inversion : media 0.78 (máx 9)
- tiene_inversion : 3261 clientes CON inversión (33.2%), 6561 SIN inversión

• segmento_edad:

segmento_edad	
Senior	5627
Adulto	3421
Joven	774

Tenencia de productos de inversión por perfil de cliente. El gráfico se calcula directamente con `pd.crosstab` sobre `tipo_inversion` y `num_prod_inversion` (sin valores fijados a mano). Solo cuentan los productos de la categoría Inversion_Ahorro.

Resultado:

- **Pensionado:** 36,1 % de sus clientes tiene al menos un producto de inversión (2.899 de 8.034).
- **Juvenil:** 25,8 % (200 de 774).
- **Conservador** (19–30 años): **16,0 %** (162 de 1.014). Es el perfil que **menos invierte**, tanto en porcentaje como en número de productos, lo que refuerza la oportunidad del producto de inversión para la primera vivienda.

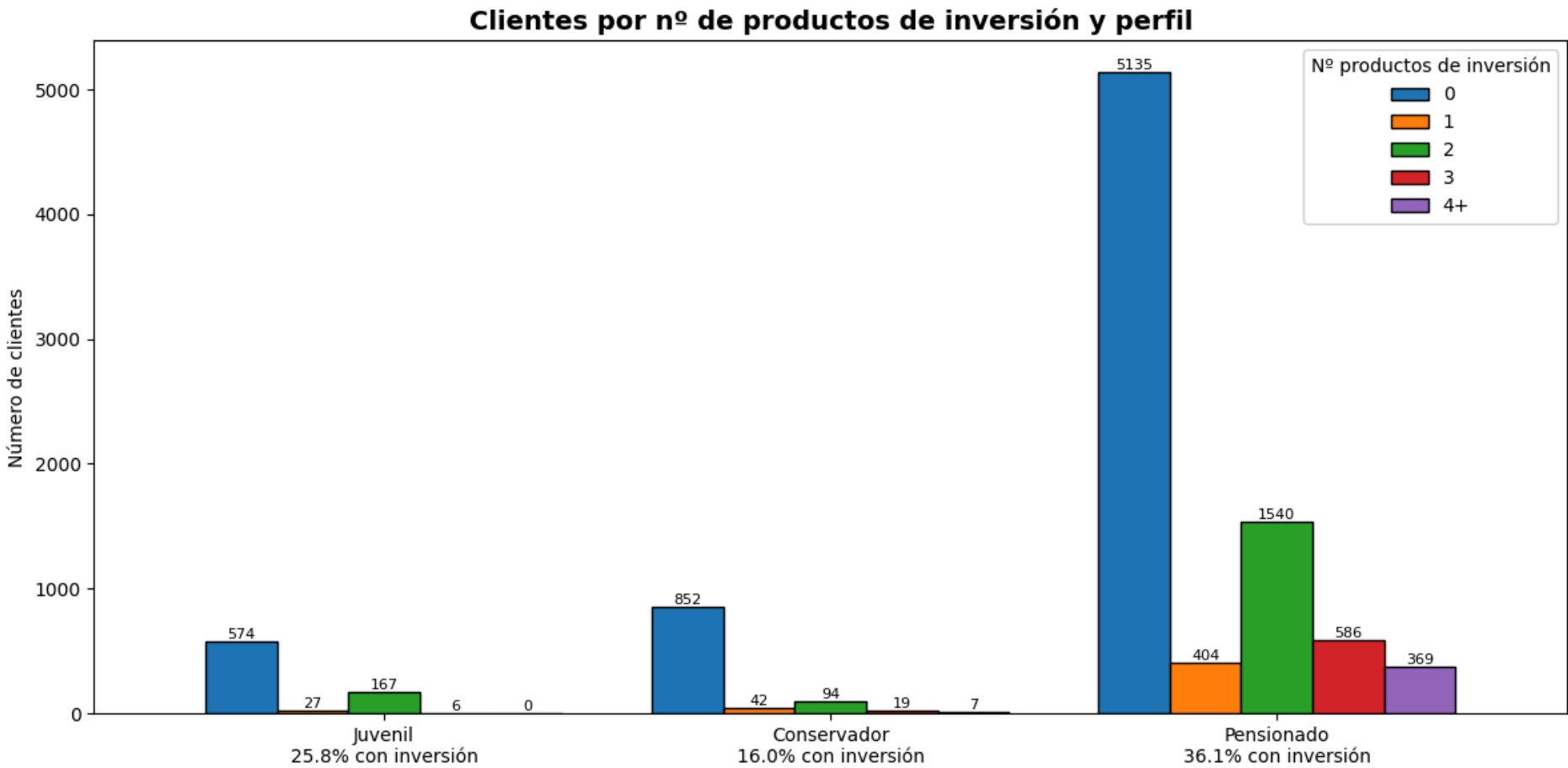
La versión anterior de este gráfico usaba porcentajes fijados a mano y concluía que el perfil Juvenil era el que menos invertía. Con los datos reales, el perfil con menor tenencia es el Conservador.

In [144...

```
# Tabla cruzada real: perfil de inversión x nº de productos de inversión (0, 1, 2, 3, 4+)
orden_perfiles = ['Inversión Perfil Juvenil', 'Inversión Perfil Conservador', 'Inversión Perfil Pensionado']
tramos = pd.cut(df_datasabadell['num_prod_inversion'], bins=[-1, 0, 1, 2, 3, np.inf],
                labels=['0', '1', '2', '3', '4+'])
tabla_inv = (pd.crosstab(df_datasabadell['tipo_inversion'], tramos)
              .reindex(index=orden_perfiles, columns=['0', '1', '2', '3', '4+'], fill_value=0))
tabla_inv.columns = tabla_inv.columns.astype(str)
tabla_inv['TOTAL'] = tabla_inv.sum(axis=1)
tabla_inv['% con inversión'] = ((1 - tabla_inv['0'] / tabla_inv['TOTAL']) * 100).round(1)

fig, ax = plt.subplots(figsize=(12, 6))
tabla_inv[['0', '1', '2', '3', '4+']].plot(kind='bar', ax=ax, edgecolor='black', width=0.8)
ax.set_title('Clientes por nº de productos de inversión y perfil', fontsize=14, fontweight='bold')
ax.set_xlabel('')
ax.set_ylabel('Número de clientes')
ax.set_xticklabels([f"{p.replace('Inversión Perfil ', '')}\n{n{pct}% con inversión}"
                    for p, pct in zip(tabla_inv.index, tabla_inv['% con inversión'])], rotation=0)
ax.legend(title='Nº productos de inversión')
for cont in ax.containers:
    ax.bar_label(cont, fmt='%d', fontsize=8)
plt.tight_layout()
plt.show()

tabla_inv
```



Out[144...

num_prod_inversion	0	1	2	3	4+	TOTAL	% con inversión
tipo_inversion							
Inversión Perfil Juvenil	574	27	167	6	0	774	25.8
Inversión Perfil Conservador	852	42	94	19	7	1014	16.0
Inversión Perfil Pensionado	5135	404	1540	586	369	8034	36.1

4. INGESTA DE LOS DATOS EN MySQL

El dataset limpio (df_datasabade11 , 9.822 clientes) se prepara y se inserta en MySQL desde este notebook.

Paso	Qué se hace	Resultado
4.1 Conexión	SQLAlchemy + pymysql ; crea la base de datos si no existe	engine
4.2 Preparación	gastos_tot de texto moneda a numérico, tipos compatibles con MySQL, validación de claves	df_stg
4.3 Carga	Insertión en la tabla de staging y clave primaria id_cliente	stg_clientes_original
4.4 Verificación	Recuento de filas en MySQL frente al DataFrame	9.822 = 9.822

Requisitos: pip install sqlalchemy pymysql y un servidor MySQL en ejecución. Las credenciales se leen de variables de entorno (MYSQL_USER , MYSQL_PASSWORD , MYSQL_HOST , MYSQL_PORT , MYSQL_DB); si no hay contraseña definida, el notebook la pide por pantalla.

In [145...

```
# [CELDA_CONEXION_MYSQL]
# 4.1 Conexión a MySQL
import os
import getpass
from sqlalchemy import create_engine, text
from sqlalchemy.engine import URL

# === DICCIONARIO CORREGIDO ===
MYSQL_CONFIG = {
    'usuario': os.getenv('MYSQL_USER', 'root'),
    'password': os.getenv('MYSQL_PASSWORD') or getpass.getpass('Contraseña de MySQL: '),
    'host': os.getenv('MYSQL_HOST', 'localhost'),
    'puerto': int(os.getenv('MYSQL_PORT', '3306')),
    'base_datos': os.getenv('MYSQL_DB', 'bancosabadell_dw'),
}
```

```
# === FUNCIÓN url_mysql CORREGIDA ===
def url_mysql(con_base_datos=True):
    # VERIFICACIÓN DE SEGURIDAD
    claves_requeridas = ['usuario', 'password', 'host', 'puerto', 'base_datos']
    for clave in claves_requeridas:
        if clave not in MYSQL_CONFIG:
            raise KeyError(f"Clave '{clave}' no encontrada en MYSQL_CONFIG")

    return URL.create(
        'mysql+pymysql',
        username=MYSQL_CONFIG['usuario'],          # ← CORREGIDO
        password=MYSQL_CONFIG['password'],         # ← CORREGIDO
        host=MYSQL_CONFIG['host'],
        port=MYSQL_CONFIG['puerto'],
        database=MYSQL_CONFIG['base_datos'] if con_base_datos else None,
        query={'charset': 'utf8mb4'},
    )

# === PRUEBA DE CONEXIÓN ===
print("🐞 Probando conexión...")
try:
    # Probar que la función funciona
    url_prueba = url_mysql(con_base_datos=False)
    print(f"✅ url_mysql() funciona correctamente")

    # Crear engine_servidor
    engine_servidor = create_engine(url_mysql(con_base_datos=False))
    print(f"✅ engine_servidor creado exitosamente")

    # Crear la base de datos si no existe
    with engine_servidor.begin() as con:
        con.execute(text(
            f"CREATE DATABASE IF NOT EXISTS `{MYSQL_CONFIG['base_datos']}` "
            "CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci"
        ))
    engine_servidor.dispose()
    print(f"✅ Base de datos '{MYSQL_CONFIG['base_datos']}' verificada/creada")

    # Crear engine principal
    engine = create_engine(url_mysql(), pool_pre_ping=True)

    # Verificar conexión
    import pandas as pd
    pd.read_sql(text("SELECT 1 AS prueba"), engine) # comprueba que la conexión responde
    print(f"✅ Conectado a MySQL | base de datos: {MYSQL_CONFIG['base_datos']}")

except KeyError as e:
    print(f"❌ ERROR DE CLAVES: {e}")
    print("  Verifica que MYSQL_CONFIG tenga las claves correctas:")
    print(f"  Claves disponibles: {list(MYSQL_CONFIG.keys())}")
    raise
except Exception as e:
    print(f"❌ ERROR GENERAL: {e}")
    print("  Posibles causas:")
    print("    1. MySQL no está corriendo en localhost:3306")
    print("    2. Usuario/contraseña incorrectos")
    print("    3. No tienes permisos para crear bases de datos")
    raise
```

🐞 Probando conexión...
✅ url_mysql() funciona correctamente
✅ engine_servidor creado exitosamente
✅ Base de datos 'bancosabadell_dw' verificada/creada
✅ Conectado a MySQL | base de datos: bancosabadell_dw

4.2 Preparación del dataset para la ingesta

Transformaciones de limpieza necesarias para que los datos entren en columnas tipadas de MySQL:

- gastos_tot : de texto con formato moneda (" € 1,278.64" , " € - ") a número; el guion equivale a 0 €. Dos celdas del Excel de origen contienen ##### (desbordamiento de celda: el importe no se conserva) y se cargan como nulo.
- Columnas categóricas y de fecha convertidas a tipos nativos (texto y datetime).
- Indicadores ind_* y flags garantizados como enteros 0/1.
- Validaciones: id_cliente único y sin nulos, sin códigos de provincia desconocidos.

In [146...

```
def convertir_moneda(serie):
    """ ' € 1,278.64' -> 1278.64 ; ' € - ' -> 0.0 """
    limpio = (serie.astype(str)
               .str.replace('€', '', regex=False)
               .str.replace(',', '', regex=False)
               .str.strip())

    limpio = limpio.replace({'-': '0', '': np.nan, 'nan': np.nan})
    # '#####' = desbordamiento de celda en el Excel de origen: el importe se perdió -> nulo
    limpio = limpio.mask(limpio.str.fullmatch(r'#+', na=False))
    return pd.to_numeric(limpio, errors='coerce')

def preparar_para_mysql(df):
    out = df.copy()

    # 1) gastos_tot: texto moneda -> numérico
    if not pd.api.types.is_numeric_dtype(out['gastos_tot']):
        desbordados = out['gastos_tot'].astype(str).str.strip().str.fullmatch(r'#+').sum()
        out['gastos_tot'] = convertir_moneda(out['gastos_tot'])
        print(f"gastos_tot: {desbordados} valores '#####' en el Excel de origen se cargan como nulo")

    # 2) tipos nativos
    for col in out.select_dtypes(include='category').columns:
        out[col] = out[col].astype(str)
    out['fecha_apertura'] = pd.to_datetime(out['fecha_apertura'])

    # 3) indicadores y flags como enteros 0/1
    flags = IND_COLS + ['tiene_inversion', 'renta_imputada', 'es_extranjero', 'residente', 'tpv']
    out[flags] = out[flags].fillna(0).astype(int)

    # 4) validaciones
```

```
assert out['id_cliente'].notna().all(), "Hay id_cliente nulos"
assert out['id_cliente'].is_unique, "Hay id_cliente duplicados"
assert out['provincia_nombre'].notna().all(), "Hay códigos de provincia no reconocidos"
assert out['gastos_tot'].isna().sum() <= 5, "gastos_tot tiene demasiados valores no convertibles"
return out
```

```
df_stg = preparar_para_mysql(df_databadell)
print(f"gastos_tot convertido: media {df_stg['gastos_tot'].mean():.2f} € | "
      f"clientes con gasto 0: {(df_stg['gastos_tot'] == 0).sum()}")
print(f"Dataset listo para ingesta: {df_stg.shape[0]} filas x {df_stg.shape[1]} columnas")
```

gastos_tot: 2 valores '#####' en el Excel de origen se cargan como nulo
gastos_tot convertido: media 251.95 € | clientes con gasto 0: 6522
Dataset listo para ingesta: 9822 filas x 96 columnas

In [147...

```
# 4.3 Carga en La tabla de staging (copia fiel del dataset limpio) + clave primaria
TABLA_STAGING = 'stg_clientes_original'

df_stg.to_sql(TABLA_STAGING, engine, if_exists='replace', index=False, chunksize=1000, method='multi')
with engine.begin() as con:
    con.execute(text(f"ALTER TABLE {TABLA_STAGING} ADD PRIMARY KEY (id_cliente)"))

# 4.4 Verificación de la ingesta
control = pd.read_sql(text(
    f"SELECT COUNT(*) AS filas, COUNT(DISTINCT id_cliente) AS clientes_unicos, "
    f"ROUND(SUM(totalpasta), 2) AS patrimonio_total FROM {TABLA_STAGING}"
), engine)
print(control.to_string(index=False))
assert control.loc[0, 'filas'] == len(df_stg), "El nº de filas en MySQL no coincide con el DataFrame"
assert abs(control.loc[0, 'patrimonio_total'] - df_stg['totalpasta'].sum()) < 1, "El patrimonio total no cuadra"
print(f"\n✅ Ingesta verificada: {len(df_stg)} clientes en {TABLA_STAGING}")
```

```
filas    clientes_unicos    patrimonio_total
9822              9822          231076038.57
```

✅ Ingesta verificada: 9822 clientes en stg_clientes_original

5. TRANSFORMACIÓN Y CARGA: MODELO DEL DATAWAREHOUSE

El proceso **extrae** los clientes de la tabla de staging en MySQL, los **transforma** con pandas y **carga** el resultado en tablas nuevas del DWH.

Transformaciones implementadas

#	Transformación	Tipo	Tabla destino
T1	Decodificación de nivren a nivel_renta (Bajo ... Alto)	Mapeo / enriquecimiento	dim_cliente
T2	Provincia → nombre, comunidad autónoma y flag extranjero	Enriquecimiento con tabla maestra	dim_geografia
T3	Catálogo de 45 productos con nombre y categoría	Tabla maestra	dim_producto
T4	Calendario (año, trimestre, mes, día de la semana, fin de semana)	Derivación de fechas	dim_tiempo
T5	Claves subrogadas (*_sk) y resolución de claves foráneas	Integración (joins)	todas
T6	Renombrado de columnas técnicas a nombres de negocio	Estandarización	fact_cliente_snapshot
T7	deuda_total , ratio_endeudamiento y nivel_riesgo_endeudamiento	Cálculo	fact_cliente_snapshot
T8	Flag tiene_datos_familia para los 4.890 clientes sin datos familiares	Calidad de datos	dim_cliente
T9	Pivotado de 45 columnas ind_* a filas cliente-producto (melt)	Reestructuración	fact_producto_cliente
T10	Agregados por segmento, nivel de renta, provincia, producto y nº de productos	Agregación (groupby)	agg_*
T11	Ranking de los 100 clientes con más patrimonio y su peso en la cartera	Ranking / ventana	agg_top_clientes_patrimonio

Modelo: dos tablas de hechos (fact_cliente_snapshot , fact_producto_cliente) que comparten las dimensiones dim_cliente y dim_tiempo , con dim_segmento y dim_geografia colgando de dim_cliente . Sobre ellas se construyen 6 tablas agregadas para la explotación y 3 tablas de KPIs (sección 6).

In [148...

```
# 5.1 Extracción desde MySQL (staging)
df_base = pd.read_sql(text(f"SELECT * FROM {TABLA_STAGING}"), engine)
df_base['fecha_apertura'] = pd.to_datetime(df_base['fecha_apertura'])
print(f"Extraídos {len(df_base)} clientes de {TABLA_STAGING}")
```

Extraídos 9822 clientes de stg_clientes_original

5.2 Creación del modelo en MySQL (DDL)

Se crean las tablas con sus **PK, FK** y restricciones **UNIQUE**. El script se exporta también a output/modelo_dwh_mysql.sql para entregarlo junto al modelo, igual que en la Entrega 1.

RECREAR_MODELO = True borra y vuelve a crear todas las tablas, **incluido el histórico de KPIs**. Con False (valor por defecto) solo se crean las que no existen, así que el histórico de ejecuciones se conserva.

In [149...

```
RECREAR_MODELO = False

TABLAS_DWH = [ # orden de creación / carga (padres antes que hijos)
    'dim_segmento', 'dim_geografia', 'dim_producto', 'dim_tiempo', 'dim_cliente',
    'fact_cliente_snapshot', 'fact_producto_cliente',
    'agg_segmento', 'agg_nivel_renta', 'agg_provincia', 'agg_producto',
    'agg_distribucion_productos', 'agg_top_clientes_patrimonio',
    'kpi_definicion', 'kpi_ejecucion', 'kpi_resultado',
]

SIO_COLS = ['sio_accionabilidad', 'sio_ahorro', 'sio_aldia', 'sio_captacion', 'sio_desvinculacion',
            'sio_financiacion', 'sio_mediospago', 'sio_noaccionable', 'sio_proteccion',
            'sio_transaccionalidad', 'sio_vinculacion']

_sio_ddl = ",\n".join(f"{c} DECIMAL(8,6)" for c in SIO_COLS)
_tabla = "ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish_ci"

DDL_DWH = [
f"""CREATE TABLE IF NOT EXISTS dim_segmento (
    segmento_sk      INT NOT NULL,
    tipo_inversion   VARCHAR(50) NOT NULL,
    descripcion      VARCHAR(300),
    edad_min         SMALLINT,
```

```
    edad_max          SMALLINT,
    orden_perfil      TINYINT,
    CONSTRAINT pk_dim_segmento PRIMARY KEY (segmento_sk),
    CONSTRAINT uq_dim_segmento UNIQUE (tipo_inversion)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS dim_geografia (
    geografia_sk      INT NOT NULL,
    prov_codigo       SMALLINT NOT NULL,
    provincia_nombre  VARCHAR(60) NOT NULL,
    comunidad_autonoma VARCHAR(60),
    es_extranjero     TINYINT(1) NOT NULL,
    CONSTRAINT pk_dim_geografia PRIMARY KEY (geografia_sk),
    CONSTRAINT uq_dim_geografia UNIQUE (prov_codigo)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS dim_producto (
    producto_sk       INT NOT NULL,
    codigo_indicador  VARCHAR(50) NOT NULL,
    nombre_producto   VARCHAR(150) NOT NULL,
    categoria         VARCHAR(30) NOT NULL,
    es_inversion      TINYINT(1) NOT NULL,
    es_indicador_resumen TINYINT(1) NOT NULL,
    CONSTRAINT pk_dim_producto PRIMARY KEY (producto_sk),
    CONSTRAINT uq_dim_producto UNIQUE (codigo_indicador)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS dim_tiempo (
    tiempo_id         INT NOT NULL,
    fecha             DATE NOT NULL,
    anio              SMALLINT NOT NULL,
    trimestre         TINYINT NOT NULL,
    mes               TINYINT NOT NULL,
    dia               TINYINT NOT NULL,
    dia_semana        TINYINT NOT NULL,
    nombre_mes        VARCHAR(20) NOT NULL,
    nombre_dia_semana VARCHAR(20) NOT NULL,
    es_fin_semana     TINYINT(1) NOT NULL,
    CONSTRAINT pk_dim_tiempo PRIMARY KEY (tiempo_id),
    CONSTRAINT uq_dim_tiempo UNIQUE (fecha)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS dim_cliente (
    cliente_sk        INT NOT NULL,
    id_cliente        INT NOT NULL,
    grupo_edad        VARCHAR(20),
    nivren_codigo     TINYINT,
    nivel_renta       VARCHAR(20),
    residente         TINYINT(1),
    tiene_datos_familia TINYINT(1) NOT NULL,
    tipo_familia      VARCHAR(100),
    miembros_familia  TINYINT,
    menores_18        TINYINT,
    entre_18_45        TINYINT,
    entre_46_65        TINYINT,
    jubilados_familia TINYINT,
    fase_cliente      SMALLINT,
    tipcart_codigo    SMALLINT,
    fecha_apertura    DATE,
    grupo_antiguedad  VARCHAR(20),
    geografia_sk      INT NOT NULL,
    segmento_sk       INT NOT NULL,
    fecha_carga       DATETIME NOT NULL,
    CONSTRAINT pk_dim_cliente PRIMARY KEY (cliente_sk),
    CONSTRAINT uq_dim_cliente UNIQUE (id_cliente),
    CONSTRAINT fk_cliente_geografia FOREIGN KEY (geografia_sk) REFERENCES dim_geografia (geografia_sk),
    CONSTRAINT fk_cliente_segmento FOREIGN KEY (segmento_sk) REFERENCES dim_segmento (segmento_sk)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS fact_cliente_snapshot (
    fact_id           BIGINT NOT NULL,
    cliente_sk        INT NOT NULL,
    tiempo_id         INT NOT NULL,
    tiempo_apertura_id INT NOT NULL,
    edad              SMALLINT,
    antiguedad_meses  SMALLINT,
    total_patrimonio  DECIMAL(15,2),
    importe_posicion_deudora DECIMAL(15,2),
    importe_posicion_acreadora DECIMAL(15,2),
    renta_final       DECIMAL(15,2),
    renta_imputada    TINYINT(1) NOT NULL,
    gastos_totales    DECIMAL(15,2),
    margen_comercial  DECIMAL(15,2),
    credito_utilizado DECIMAL(15,2),
    revolving_utilizado DECIMAL(15,2),
    total_hipoteca    DECIMAL(15,2),
    deuda_total       DECIMAL(15,2),
    ratio_endeudamiento DECIMAL(18,4),
    nivel_riesgo_endeudamiento VARCHAR(20),
    nomina_importe    DECIMAL(15,2),
    rdom              INT,
    tpv               TINYINT(1),
    total_proteccion  TINYINT,
    {_sio_ddl},
    total_productos_activos TINYINT NOT NULL,
    num_prod_inversion TINYINT NOT NULL,
    tiene_inversion    TINYINT(1) NOT NULL,
    fecha_carga       DATETIME NOT NULL,
    CONSTRAINT pk_fact_cliente_snapshot PRIMARY KEY (fact_id),
    CONSTRAINT uq_fact_cliente_snapshot UNIQUE (cliente_sk, tiempo_id),
    CONSTRAINT fk_fcs_cliente FOREIGN KEY (cliente_sk) REFERENCES dim_cliente (cliente_sk),
    CONSTRAINT fk_fcs_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id),
    CONSTRAINT fk_fcs_apertura FOREIGN KEY (tiempo_apertura_id) REFERENCES dim_tiempo (tiempo_id)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS fact_producto_cliente (
    fact_id          BIGINT NOT NULL,
    cliente_sk       INT NOT NULL,
    producto_sk      INT NOT NULL,
```

```
tiempo_id INT NOT NULL,
CONSTRAINT pk_fact_producto_cliente PRIMARY KEY (fact_id),
CONSTRAINT uq_fact_producto_cliente UNIQUE (cliente_sk, producto_sk, tiempo_id),
CONSTRAINT fk_fpc_cliente FOREIGN KEY (cliente_sk) REFERENCES dim_cliente (cliente_sk),
CONSTRAINT fk_fpc_producto FOREIGN KEY (producto_sk) REFERENCES dim_producto (producto_sk),
CONSTRAINT fk_fpc_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_segmento (
tiempo_id INT NOT NULL,
segmento_sk INT NOT NULL,
total_clientes INT NOT NULL,
pct_clientes DECIMAL(7,2),
patrimonio_total DECIMAL(18,2),
patrimonio_medio DECIMAL(15,2),
margen_total DECIMAL(18,2),
margen_medio DECIMAL(15,2),
pct_margen DECIMAL(7,2),
productos_medios DECIMAL(6,2),
clientes_con_inversion INT,
pct_con_inversion DECIMAL(7,2),
ratio_endeudamiento_mediano DECIMAL(18,4),
clientes_riesgo_critico INT,
CONSTRAINT pk_agg_segmento PRIMARY KEY (tiempo_id, segmento_sk),
CONSTRAINT fk_aseg_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id),
CONSTRAINT fk_aseg_segmento FOREIGN KEY (segmento_sk) REFERENCES dim_segmento (segmento_sk)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_nivel_renta (
tiempo_id INT NOT NULL,
nivren_codigo TINYINT NOT NULL,
nivel_renta VARCHAR(20) NOT NULL,
total_clientes INT NOT NULL,
pct_clientes DECIMAL(7,2),
patrimonio_medio DECIMAL(15,2),
margen_medio DECIMAL(15,2),
productos_medios DECIMAL(6,2),
pct_con_inversion DECIMAL(7,2),
CONSTRAINT pk_agg_nivel_renta PRIMARY KEY (tiempo_id, nivren_codigo),
CONSTRAINT fk_anr_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_provincia (
tiempo_id INT NOT NULL,
geografia_sk INT NOT NULL,
total_clientes INT NOT NULL,
pct_clientes DECIMAL(7,2),
patrimonio_total DECIMAL(18,2),
patrimonio_medio DECIMAL(15,2),
margen_medio DECIMAL(15,2),
productos_medios DECIMAL(6,2),
CONSTRAINT pk_agg_provincia PRIMARY KEY (tiempo_id, geografia_sk),
CONSTRAINT fk_aprov_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id),
CONSTRAINT fk_aprov_geografia FOREIGN KEY (geografia_sk) REFERENCES dim_geografia (geografia_sk)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_producto (
tiempo_id INT NOT NULL,
producto_sk INT NOT NULL,
clientes_con_producto INT NOT NULL,
pct_penetracion DECIMAL(7,3),
CONSTRAINT pk_agg_producto PRIMARY KEY (tiempo_id, producto_sk),
CONSTRAINT fk_aprod_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id),
CONSTRAINT fk_aprod_producto FOREIGN KEY (producto_sk) REFERENCES dim_producto (producto_sk)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_distribucion_productos (
tiempo_id INT NOT NULL,
numero_productos TINYINT NOT NULL,
total_clientes INT NOT NULL,
pct_clientes DECIMAL(7,2),
patrimonio_medio DECIMAL(15,2),
margen_medio DECIMAL(15,2),
CONSTRAINT pk_agg_distribucion_productos PRIMARY KEY (tiempo_id, numero_productos),
CONSTRAINT fk_adp_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS agg_top_clientes_patrimonio (
tiempo_id INT NOT NULL,
ranking SMALLINT NOT NULL,
cliente_sk INT NOT NULL,
total_patrimonio DECIMAL(15,2),
pct_patrimonio_cartera DECIMAL(7,3),
margen_comercial DECIMAL(15,2),
total_productos_activos TINYINT,
CONSTRAINT pk_agg_top_clientes PRIMARY KEY (tiempo_id, ranking),
CONSTRAINT fk_atop_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id),
CONSTRAINT fk_atop_cliente FOREIGN KEY (cliente_sk) REFERENCES dim_cliente (cliente_sk)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS kpi_definicion (
kpi_id VARCHAR(40) NOT NULL,
nombre VARCHAR(120) NOT NULL,
descripcion VARCHAR(500),
formula VARCHAR(500),
unidad VARCHAR(30),
frecuencia VARCHAR(20),
objetivo_tfm VARCHAR(200),
tabla_origen VARCHAR(100),
CONSTRAINT pk_kpi_definicion PRIMARY KEY (kpi_id)
) {_tabla}""",

f""""CREATE TABLE IF NOT EXISTS kpi_ejecucion (
ejecucion_id CHAR(36) NOT NULL,
fecha_ejecucion DATETIME NOT NULL,
tiempo_id INT NOT NULL,
total_clientes INT NOT NULL,
CONSTRAINT pk_kpi_ejecucion PRIMARY KEY (ejecucion_id),
CONSTRAINT fk_kej_tiempo FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo (tiempo_id)
```

```
) {_tabla}""",

f"""CREATE TABLE IF NOT EXISTS kpi_resultado (
    resultado_id      BIGINT NOT NULL AUTO_INCREMENT,
    ejecucion_id      CHAR(36) NOT NULL,
    kpi_id             VARCHAR(40) NOT NULL,
    dimension          VARCHAR(40) NOT NULL,
    valor_dimension    VARCHAR(100) NOT NULL,
    metrica            VARCHAR(60) NOT NULL,
    valor              DECIMAL(20,4),
    CONSTRAINT pk_kpi_resultado PRIMARY KEY (resultado_id),
    CONSTRAINT uq_kpi_resultado UNIQUE (ejecucion_id, kpi_id, dimension, valor_dimension, metrica),
    CONSTRAINT fk_kres_ejecucion FOREIGN KEY (ejecucion_id) REFERENCES kpi_ejecucion (ejecucion_id),
    CONSTRAINT fk_kres_kpi      FOREIGN KEY (kpi_id)      REFERENCES kpi_definicion (kpi_id)
) {_tabla}""",
]

with engine.begin() as con:
    if RECREAR_MODELO:
        con.execute(text("SET FOREIGN_KEY_CHECKS = 0"))
        for tabla in reversed(TABLAS_DWH):
            con.execute(text(f"DROP TABLE IF EXISTS {tabla}"))
        con.execute(text("SET FOREIGN_KEY_CHECKS = 1"))
        for sentencia in DDL_DWH:
            con.execute(text(sentencia))

# Exportar el DDL como script .sql (entregable del modelo de datos)
ruta_sql = Path.cwd() / 'output' / 'modelo_dwh_mysql.sql'
ruta_sql.parent.mkdir(exist_ok=True)
ruta_sql.write_text(
    f"-- Modelo DWH BancoSabadell (MySQL 8) - generado desde el notebook\n"
    f"CREATE DATABASE IF NOT EXISTS {MYSQL_CONFIG['base_datos']} CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;\n"
    f"USE {MYSQL_CONFIG['base_datos']};\n\n" + ";".join(DDL_DWH) + ";\n",
    encoding='utf-8',
)
print(f"✅ {len(DDL_DWH)} tablas del DWH verificadas/creadas en MySQL")
print(f"📄 DDL exportado a: {ruta_sql}")
```

- ✅ 16 tablas del DWH verificadas/creadas en MySQL
- 📄 DDL exportado a: c:\Users\jmendozaarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 2\output\modelo_dwh_mysql.sql

5.3 Dimensiones

Cada dimensión recibe una **clave subrogada** (*_sk) generada en Python. `dim_cliente` resuelve sus claves foráneas hacia `dim_geografia` y `dim_segmento` mediante *merge* (equivalente a un JOIN).

In [150...

```
FECHA_CARGA = pd.Timestamp.now().floor('s')
TIEMPO_ID_SNAPSHOT = int(FECHA_SNAPSHOT.strftime('%Y%m%d'))

MAPA_NIVEL_RENTA = {0: 'Sin datos', 1: 'Bajo', 2: 'Medio-Bajo', 3: 'Medio', 4: 'Medio-Alto', 5: 'Alto'}
NOMBRES_MES = ['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio', 'Julio',
               'Agosto', 'Septiembre', 'Octubre', 'Noviembre', 'Diciembre']
NOMBRES_DIA = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado', 'Domingo']

# --- dim_segmento -----
dim_segmento = pd.DataFrame([
    (1, 'Inversión Perfil Juvenil', 'Clientes de hasta 18 años: producto de inversión para estudios', 0, 18, 1),
    (2, 'Inversión Perfil Conservador', 'Clientes de 19 a 30 años: producto de inversión para primera vivienda', 19, 30, 2),
    (3, 'Inversión Perfil Pensionado', 'Clientes de más de 30 años: producto de inversión para jubilación', 31, 150, 3),
], columns=['segmento_sk', 'tipo_inversion', 'descripcion', 'edad_min', 'edad_max', 'orden_perfil'])

# --- dim_geografia (T2) -----
dim_geografia = pd.DataFrame({'prov_codigo': sorted(PROVINCIAS_INE)})
dim_geografia['geografia_sk'] = range(1, len(dim_geografia) + 1)
dim_geografia['provincia_nombre'] = dim_geografia['prov_codigo'].map(PROVINCIAS_INE)
dim_geografia['comunidad_autonoma'] = dim_geografia['prov_codigo'].map(CCAA_POR_PROVINCIA)
dim_geografia['es_extranjero'] = (dim_geografia['prov_codigo'] == 99).astype(int)
dim_geografia = dim_geografia[['geografia_sk', 'prov_codigo', 'provincia_nombre', 'comunidad_autonoma', 'es_extranjero']]

# --- dim_producto (T3) -----
dim_producto = pd.DataFrame(
    [(i, cod, nom, cat, int(cat == 'Inversion_Ahorro'), int(cod in INDICADORES_RESUMEN))
     for i, (cod, (nom, cat)) in enumerate(CATALOGO_PRODUCTOS.items(), start=1)],
    columns=['producto_sk', 'codigo_indicador', 'nombre_producto', 'categoria', 'es_inversion',
             'es_indicador_resumen'])

# --- dim_tiempo (T4): fecha del snapshot + fechas de apertura -----
fechas = pd.Series(pd.to_datetime(
    list(df_base['fecha_apertura'].dropna().unique() + [FECHA_SNAPSHOT])).drop_duplicates().sort_values())
dim_tiempo = pd.DataFrame({'fecha': fechas.dt.normalize()})
dim_tiempo['tiempo_id'] = dim_tiempo['fecha'].dt.strftime('%Y%m%d').astype(int)
dim_tiempo['anio'] = dim_tiempo['fecha'].dt.year
dim_tiempo['trimestre'] = dim_tiempo['fecha'].dt.quarter
dim_tiempo['mes'] = dim_tiempo['fecha'].dt.month
dim_tiempo['dia'] = dim_tiempo['fecha'].dt.day
dim_tiempo['dia_semana'] = dim_tiempo['fecha'].dt.dayofweek + 1
dim_tiempo['nombre_mes'] = dim_tiempo['mes'].map(lambda m: NOMBRES_MES[m - 1])
dim_tiempo['nombre_dia_semana'] = dim_tiempo['dia_semana'].map(lambda d: NOMBRES_DIA[d - 1])
dim_tiempo['es_fin_semana'] = (dim_tiempo['dia_semana'] >= 6).astype(int)
dim_tiempo['fecha'] = dim_tiempo['fecha'].dt.date
dim_tiempo = dim_tiempo[['tiempo_id', 'fecha', 'anio', 'trimestre', 'mes', 'dia', 'dia_semana',
                        'nombre_mes', 'nombre_dia_semana', 'es_fin_semana']].reset_index(drop=True)

# --- dim_cliente (T1, T5, T8) -----
dim_cliente = (df_base.sort_values('id_cliente')
               .merge(dim_geografia[['geografia_sk', 'prov_codigo']], left_on='prov', right_on='prov_codigo', how='left')
               .merge(dim_segmento[['segmento_sk', 'tipo_inversion']], on='tipo_inversion', how='left'))
assert dim_cliente[['geografia_sk', 'segmento_sk']].notna().all().all(), "Clientes sin geografia o segmento"
dim_cliente['cliente_sk'] = range(1, len(dim_cliente) + 1)
dim_cliente['nivel_renta'] = dim_cliente['nivren'].map(MAPA_NIVEL_RENTA).fillna('Sin datos')
dim_cliente['tiene_datos_familia'] = dim_cliente['tipofamilia'].notna().astype(int)
dim_cliente['fecha_carga'] = FECHA_CARGA
dim_cliente['fecha_apertura'] = dim_cliente['fecha_apertura'].dt.date
dim_cliente = dim_cliente.rename(columns={
    'segmento_edad': 'grupo_edad', 'nivren': 'nivren_codigo', 'tipofamilia': 'tipo_familia',
    'miembrosfamilia': 'miembros_familia', 'menores18': 'menores_18', 'entre18_45': 'entre_18_45',
    'entre46_65': 'entre_46_65', 'jubilados': 'jubilados_familia', 'fase': 'fase_cliente',
    'tipcart': 'tipcart_codigo'})
```

```
dim_cliente = dim_cliente[[
    'cliente_sk', 'id_cliente', 'grupo_edad', 'nivren_codigo', 'nivel_renta', 'residente',
    'tiene_datos_familia', 'tipo_familia', 'miembros_familia', 'menores_18', 'entre_18_45',
    'entre_46_65', 'jubilados_familia', 'fase_cliente', 'tipcart_codigo', 'fecha_apertura',
    'grupo_antiguedad', 'geografia_sk', 'segmento_sk', 'fecha_carga']]

for nombre, tabla in [('dim_segmento', dim_segmento), ('dim_geografia', dim_geografia),
                      ('dim_producto', dim_producto), ('dim_tiempo', dim_tiempo), ('dim_cliente', dim_cliente)]:
    print(f"{nombre:<15} {len(tabla):>6} filas")
dim_cliente.head()
```

dim_segmento 3 filas
dim_geografia 53 filas
dim_producto 45 filas
dim_tiempo 495 filas
dim_cliente 9822 filas

Out[150...

	cliente_sk	id_cliente	grupo_edad	nivren_codigo	nivel_renta	residente	tiene_datos_familia	tipo_familia	miembros_familia	menores_18	entre_18_45	entre_46_65	jubilados_familia
0	1	1	Senior	2	Medio-Bajo	1	1	Resto	3.0	1.0	1.0	0.0	
1	2	2	Senior	4	Medio-Alto	1	0	NaN	NaN	NaN	NaN	NaN	NaN
2	3	3	Senior	2	Medio-Bajo	1	1	Viudos	1.0	0.0	0.0	0.0	
3	4	4	Senior	3	Medio	1	1	Familia Estandar Senior	4.0	1.0	1.0	2.0	
4	5	5	Senior	2	Medio-Bajo	1	0	NaN	NaN	NaN	NaN	NaN	NaN

5.4 Tablas de hechos

- fact_cliente_snapshot** : una fila por cliente y fecha de snapshot, con las medidas financieras renombradas a nombres de negocio (T6) y las métricas de endeudamiento calculadas (T7):
 - deuda_total = credito_utilizado + revolving_utilizado + total_hipoteca
 - ratio_endeudamiento = deuda_total / total_patrimonio (nulo si el patrimonio es 0)
 - nivel_riesgo_endeudamiento : Bajo (≤ 0,3), Moderado (≤ 0,6), Alto (≤ 1), Crítico (> 1), Sin patrimonio
- fact_producto_cliente** : una fila por cada producto que tiene contratado cada cliente. Se obtiene pivotando las 45 columnas **ind_*** a filas y quedándose con las que valen 1 (T9).

In [151...

```
def clasificar_riesgo_endeudamiento(ratio):
    niveles = pd.cut(ratio, bins=[-np.inf, 0.3, 0.6, 1.0, np.inf], labels=['Bajo', 'Moderado', 'Alto', 'Crítico'])
    return niveles.cat.add_categories('Sin patrimonio').fillna('Sin patrimonio').astype(str)

fact = df_base.merge(dim_cliente[['cliente_sk', 'id_cliente']], on='id_cliente').rename(columns={
    'totalpasta': 'total_patrimonio', 'impoped': 'importe_posicion_deudora',
    'impopeh': 'importe_posicion_acreedora', 'gastos_tot': 'gastos_totales', 'margecomercialut': 'margen_comercial',
    'credit': 'credito_utilizado', 'revolving': 'revolving_utilizado', 'tothipo': 'total_hipoteca',
    'nominaimp': 'nomina_importe', 'total_proteccio': 'total_proteccion'})
fact['tiempo_id'] = TIEMPO_ID_SNAPSHOT
fact['tiempo_apertura_id'] = fact['fecha_apertura'].dt.strftime('%Y%m%d').astype(int)

# T7: endeudamiento
fact['deuda_total'] = fact[['credito_utilizado', 'revolving_utilizado', 'total_hipoteca']].fillna(0).sum(axis=1)
fact['ratio_endeudamiento'] = (fact['deuda_total'] / fact['total_patrimonio'].replace(0, np.nan)).round(4)
fact['nivel_riesgo_endeudamiento'] = clasificar_riesgo_endeudamiento(fact['ratio_endeudamiento'])
fact['fecha_carga'] = FECHA_CARGA
fact = fact.sort_values('cliente_sk').reset_index(drop=True)
fact['fact_id'] = range(1, len(fact) + 1)

fact_cliente_snapshot = fact[[
    'fact_id', 'cliente_sk', 'tiempo_id', 'tiempo_apertura_id', 'edad', 'antiguedad_meses',
    'total_patrimonio', 'importe_posicion_deudora', 'importe_posicion_acreedora', 'renta_final',
    'renta_imputada', 'gastos_totales', 'margen_comercial', 'credito_utilizado', 'revolving_utilizado',
    'total_hipoteca', 'deuda_total', 'ratio_endeudamiento', 'nivel_riesgo_endeudamiento',
    'nomina_importe', 'rdom', 'tpv', 'total_proteccion', *SIO_COLS,
    'total_productos_activos', 'num_prod_inversion', 'tiene_inversion', 'fecha_carga']]

# T9: pivotado de 45 columnas ind_* a filas cliente-producto
fact_producto_cliente = (
    fact[['cliente_sk', *IND_COLS]]
    .melt(id_vars='cliente_sk', var_name='codigo_indicador', value_name='tiene')
    .query('tiene == 1')
    .merge(dim_producto[['producto_sk', 'codigo_indicador']], on='codigo_indicador')
    .assign(tiempo_id=TIEMPO_ID_SNAPSHOT)
    .sort_values(['cliente_sk', 'producto_sk'])
    .reset_index(drop=True)
)
fact_producto_cliente['fact_id'] = range(1, len(fact_producto_cliente) + 1)
fact_producto_cliente = fact_producto_cliente[['fact_id', 'cliente_sk', 'producto_sk', 'tiempo_id']]

# Control: el nº de filas del pivotado coincide con la suma de productos por cliente
assert len(fact_producto_cliente) == fact_cliente_snapshot['total_productos_activos'].sum()

print(f"fact_cliente_snapshot : {len(fact_cliente_snapshot):>6} filas x {fact_cliente_snapshot.shape[1]} columnas")
print(f"fact_producto_cliente : {len(fact_producto_cliente):>6} filas (cliente-producto)")
print("\nClientes por nivel de riesgo de endeudamiento:")
print(fact_cliente_snapshot['nivel_riesgo_endeudamiento'].value_counts().to_string())
```

fact_cliente_snapshot : 9822 filas x 38 columnas
fact_producto_cliente : 21260 filas (cliente-producto)

Clientes por nivel de riesgo de endeudamiento:
nivel_riesgo_endeudamiento
Bajo 7924
Crítico 1425
Sin patrimonio 347
Moderado 66
Alto 60

5.5 Tablas agregadas para la explotación (T10, T11)

Tablas precalculadas a partir de las tablas de hechos y las dimensiones. Son la base de los KPIs y de los cuadros de mando.

Tabla	Grano (PK)	Contenido
agg_segmento	snapshot + segmento	clientes, patrimonio, margen, % del margen, productos, % con inversión, endeudamiento
agg_nivel_renta	snapshot + nivel de renta	clientes, patrimonio, margen, productos medios, % con inversión
agg_provincia	snapshot + provincia	clientes, patrimonio total y medio, margen, productos
agg_producto	snapshot + producto	clientes con el producto y % de penetración
agg_distribucion_productos	snapshot + nº de productos	clientes, patrimonio y margen medio
agg_top_clientes_patrimonio	snapshot + ranking	los 100 clientes con más patrimonio y su peso en la cartera

In [152...

```
base = (fact_cliente_snapshot
        .merge(dim_cliente[['cliente_sk', 'nivren_codigo', 'nivel_renta', 'geografia_sk', 'segmento_sk']], on='cliente_sk'))
N = len(base)
margen_cartera = base['margen_comercial'].sum()
patrimonio_cartera = base['total_patrimonio'].sum()

# agg_segmento
agg_segmento = (base.groupby('segmento_sk')
                .agg(total_clientes=('cliente_sk', 'count'),
                    patrimonio_total=('total_patrimonio', 'sum'),
                    patrimonio_medio=('total_patrimonio', 'mean'),
                    margen_total=('margen_comercial', 'sum'),
                    margen_medio=('margen_comercial', 'mean'),
                    productos_medios=('total_productos_activos', 'mean'),
                    clientes_con_inversion=('tiene_inversion', 'sum'),
                    ratio_endeudamiento_mediano=('ratio_endeudamiento', 'median'),
                    clientes_riesgo_critico=('nivel_riesgo_endeudamiento', lambda s: (s == 'Crítico').sum()))
                .reset_index())
agg_segmento['pct_clientes'] = agg_segmento['total_clientes'] / N * 100
agg_segmento['pct_margen'] = agg_segmento['margen_total'] / margen_cartera * 100
agg_segmento['pct_con_inversion'] = agg_segmento['clientes_con_inversion'] / agg_segmento['total_clientes'] * 100

# agg_nivel_renta
agg_nivel_renta = (base.groupby(['nivren_codigo', 'nivel_renta'])
                  .agg(total_clientes=('cliente_sk', 'count'),
                      patrimonio_medio=('total_patrimonio', 'mean'),
                      margen_medio=('margen_comercial', 'mean'),
                      productos_medios=('total_productos_activos', 'mean'),
                      pct_con_inversion=('tiene_inversion', 'mean'))
                  .reset_index())
agg_nivel_renta['pct_clientes'] = agg_nivel_renta['total_clientes'] / N * 100
agg_nivel_renta['pct_con_inversion'] *= 100

# agg_provincia
agg_provincia = (base.groupby('geografia_sk')
                 .agg(total_clientes=('cliente_sk', 'count'),
                     patrimonio_total=('total_patrimonio', 'sum'),
                     patrimonio_medio=('total_patrimonio', 'mean'),
                     margen_medio=('margen_comercial', 'mean'),
                     productos_medios=('total_productos_activos', 'mean'))
                 .reset_index())
agg_provincia['pct_clientes'] = agg_provincia['total_clientes'] / N * 100

# agg_producto (incluye los productos sin ningún cliente, con 0)
agg_producto = (dim_producto[['producto_sk']]
                .merge(fact_producto_cliente.groupby('producto_sk').size().rename('clientes_con_producto').reset_index(),
                      on='producto_sk', how='left')
                .fillna({'clientes_con_producto': 0}))
agg_producto['clientes_con_producto'] = agg_producto['clientes_con_producto'].astype(int)
agg_producto['pct_penetracion'] = agg_producto['clientes_con_producto'] / N * 100

# agg_distribucion_productos
agg_distribucion_productos = (base.groupby('total_productos_activos')
                              .agg(total_clientes=('cliente_sk', 'count'),
                                  patrimonio_medio=('total_patrimonio', 'mean'),
                                  margen_medio=('margen_comercial', 'mean'))
                              .reset_index().rename(columns={'total_productos_activos': 'numero_productos'}))
agg_distribucion_productos['pct_clientes'] = agg_distribucion_productos['total_clientes'] / N * 100

# agg_top_clientes_patrimonio (T11)
agg_top_clientes_patrimonio = (base.nlargest(100, 'total_patrimonio')
                               [['cliente_sk', 'total_patrimonio', 'margen_comercial', 'total_productos_activos']]
                               .reset_index(drop=True))
agg_top_clientes_patrimonio.insert(0, 'ranking', range(1, len(agg_top_clientes_patrimonio) + 1))
agg_top_clientes_patrimonio['pct_patrimonio_cartera'] = agg_top_clientes_patrimonio['total_patrimonio'] / patrimonio_cartera * 100

tablas_agregadas = {
    'agg_segmento': agg_segmento, 'agg_nivel_renta': agg_nivel_renta, 'agg_provincia': agg_provincia,
    'agg_producto': agg_producto, 'agg_distribucion_productos': agg_distribucion_productos,
    'agg_top_clientes_patrimonio': agg_top_clientes_patrimonio,
}
for nombre, tabla in tablas_agregadas.items():
    tabla.insert(0, 'tiempo_id', TIEMPO_ID_SNAPSHOT)
    tablas_agregadas[nombre] = tabla.round(4)
    print(f"{nombre:<30} {len(tabla):>4} filas")

agg_segmento          3 filas
agg_nivel_renta       6 filas
agg_provincia         53 filas
agg_producto          45 filas
agg_distribucion_productos 12 filas
agg_top_clientes_patrimonio 100 filas
```

5.6 Carga en MySQL

Carga completa (*full refresh*) del snapshot: se vacían las dimensiones, hechos y agregados y se insertan en orden padre → hijo para respetar las FK. `dim_tiempo` se carga de forma **incremental** (solo fechas nuevas) porque el histórico de KPIs la referencia. Las tablas `kpi_*` no se vacían nunca.

In [153...

```
TABLAS_A_CARGAR = {
    'dim_segmento': dim_segmento, 'dim_geografia': dim_geografia, 'dim_producto': dim_producto,
    'dim_cliente': dim_cliente, 'fact_cliente_snapshot': fact_cliente_snapshot,
    'fact_producto_cliente': fact_producto_cliente, **tablas_agregadas,
}

# 1) Vaciar (hijos antes que padres)
with engine.begin() as con:
    con.execute(text("SET FOREIGN_KEY_CHECKS = 0"))
    for tabla in reversed(list(TABLAS_A_CARGAR)):
        con.execute(text(f"DELETE FROM {tabla}"))
    con.execute(text("SET FOREIGN_KEY_CHECKS = 1"))

# 2) dim_tiempo incremental: solo las fechas que aún no existen
existentes = set(pd.read_sql(text("SELECT tiempo_id FROM dim_tiempo"), engine)['tiempo_id'])
dim_tiempo_nuevas = dim_tiempo[~dim_tiempo['tiempo_id'].isin(existentes)]
dim_tiempo_nuevas.to_sql('dim_tiempo', engine, if_exists='append', index=False, chunksize=1000, method='multi')
print(f"{'dim_tiempo':<30} {len(dim_tiempo_nuevas):>6} filas nuevas")

# 3) Resto de tablas (padres antes que hijos)
for tabla, df_tabla in TABLAS_A_CARGAR.items():
    df_tabla.to_sql(tabla, engine, if_exists='append', index=False, chunksize=1000, method='multi')
    print(f"{tabla:<30} {len(df_tabla):>6} filas cargadas")

print("\n✅ Carga del DWH completada")
```

dim_tiempo	0	filas nuevas
dim_segmento	3	filas cargadas
dim_geografia	53	filas cargadas
dim_producto	45	filas cargadas
dim_cliente	9822	filas cargadas
fact_cliente_snapshot	9822	filas cargadas
fact_producto_cliente	21260	filas cargadas
agg_segmento	3	filas cargadas
agg_nivel_renta	6	filas cargadas
agg_provincia	53	filas cargadas
agg_producto	45	filas cargadas
agg_distribucion_productos	12	filas cargadas
agg_top_clientes_patrimonio	100	filas cargadas

✅ Carga del DWH completada

6. KPIs

Los KPIs se calculan **leyendo del DWH en MySQL** (tablas de hechos, dimensiones y agregados) y su resultado se guarda en tablas, siguiendo la recomendación del enunciado:

Tabla	Contenido	Clave
kpi_definicion	Catálogo de KPIs: nombre, descripción, fórmula, unidad, frecuencia, objetivo del TFM y tabla origen	kpi_id
kpi_ejecucion	Una fila por ejecución: fecha, snapshot y nº de clientes	ejecucion_id (UUID)
kpi_resultado	Valores obtenidos en cada ejecución, en formato largo (KPI, dimensión, valor de la dimensión, métrica, valor)	resultado_id ; UNIQUE (ejecución, KPI, dimensión, valor, métrica)

Cada vez que se ejecuta el notebook se añade una nueva ejecución, así que en `kpi_resultado` queda el **histórico** de resultados. El valor global de cada KPI se guarda con `dimension = 'TOTAL'`.

In [154...

```
import uuid

KPI_DEFINICIONES = pd.DataFrame([
    dict(kpi_id='KPI_DISTRIBUCION_PRODUCTOS', nombre='Distribución de clientes por nº de productos',
        descripcion='Clientes según cuántos productos tienen; identifica mono-producto (riesgo de fuga) frente a muy vinculados.',
        formula='numero_productos = SUM(45 ind_*); clientes, % y patrimonio/margen medio por numero_productos; % mono-producto = clientes con 1 producto / total',
        unidad='clientes / %', frecuencia='Mensual', objetivo_tfm='Obj. 1 - vinculación y tipo de cuenta',
        tabla_origen='agg_distribucion_productos'),
    dict(kpi_id='KPI_ENDEUDAMIENTO', nombre='Ratio de endeudamiento',
        descripcion='Nivel de apalancamiento del cliente respecto a su patrimonio; detecta clientes en riesgo de sobreendeudamiento.',
        formula='(credito_utilizado + revolving_utilizado + total_hipoteca) / total_patrimonio; % clientes con ratio > 1',
        unidad='ratio / %', frecuencia='Mensual', objetivo_tfm='Obj. 2 - perfil financiero del cliente',
        tabla_origen='fact_cliente_snapshot'),
    dict(kpi_id='KPI_POR_NIVEL_RENTA', nombre='Cartera por nivel de renta',
        descripcion='Concentración, rentabilidad y vinculación de la cartera según el nivel socioeconómico.',
        formula='Por nivel_renta: COUNT clientes, AVG patrimonio, AVG margen, AVG productos, % con inversión',
        unidad='clientes / € / nº', frecuencia='Mensual', objetivo_tfm='Obj. 2 - perfil y necesidades del cliente',
        tabla_origen='agg_nivel_renta'),
    dict(kpi_id='KPI_POR_SEGMENTO', nombre='Comparativa de perfiles de inversión',
        descripcion='Compara Juvenil, Conservador y Pensionado en tamaño, patrimonio, margen, vinculación e inversión.',
        formula='Por tipo_inversion: COUNT, % clientes, AVG patrimonio, AVG margen, % del margen total, AVG productos, % con inversión',
        unidad='clientes / € / %', frecuencia='Mensual', objetivo_tfm='Obj. 2 y 3 - productos de inversión por edad',
        tabla_origen='agg_segmento'),
    dict(kpi_id='KPI_PENETRACION_PRODUCTO', nombre='Penetración de producto',
        descripcion='Grado de adopción de cada producto y categoría sobre el total de clientes.',
        formula='clientes_con_producto / total_clientes x 100, por producto y por categoría',
        unidad='%', frecuencia='Mensual', objetivo_tfm='Obj. 1 - tipo de cuenta con más y menos clientes',
        tabla_origen='agg_producto'),
    dict(kpi_id='KPI_POR_PROVINCIA', nombre='Cartera por provincia',
        descripcion='Distribución territorial de clientes, patrimonio y margen.',
        formula='Por provincia: COUNT clientes, SUM y AVG patrimonio, AVG margen',
        unidad='clientes / €', frecuencia='Mensual', objetivo_tfm='Obj. 2 - segmentación de clientes',
        tabla_origen='agg_provincia'),
    dict(kpi_id='KPI_TOP_CLIENTES_PATRIMONIO', nombre='Concentración de patrimonio en el Top 100',
        descripcion='Peso de los 100 clientes con más patrimonio sobre el total de la cartera.',
        formula='SUM(patrimonio top 100) / SUM(patrimonio total) x 100',
        unidad='%', frecuencia='Mensual', objetivo_tfm='Obj. 2 - clientes de mayor valor',
        tabla_origen='agg_top_clientes_patrimonio'),
])

# Alta o actualización del catálogo de KPIs (sin borrar: el histórico lo referencia)
existentes = set(pd.read_sql(text("SELECT kpi_id FROM kpi_definicion"), engine)['kpi_id'])
nuevos = KPI_DEFINICIONES[~KPI_DEFINICIONES['kpi_id'].isin(existentes)]
nuevos.to_sql('kpi_definicion', engine, if_exists='append', index=False)
with engine.begin() as con:
    for fila in KPI_DEFINICIONES[KPI_DEFINICIONES['kpi_id'].isin(existentes)].to_dict('records'):
        con.execute(text(
            "UPDATE kpi_definicion SET nombre = :nombre, descripcion = :descripcion, formula = :formula, "
```

```
        "unidad = :unidad, frecuencia = :frecuencia, objetivo_tfm = :objetivo_tfm, "
        "tabla_origen = :tabla_origen WHERE kpi_id = :kpi_id"), fila)

# Registro de la ejecución
EJECUCION_ID = str(uuid.uuid4())
pd.DataFrame([{'ejecucion_id': EJECUCION_ID, 'fecha_ejecucion': pd.Timestamp.now().floor('s'),
               'tiempo_id': TIEMPO_ID_SNAPSHOT, 'total_clientes': len(fact_cliente_snapshot),
               }]).to_sql('kpi_ejecucion', engine, if_exists='append', index=False)

resultados_kpi = []

def registrar(kpi_id, metrica, valor, dimension='TOTAL', valor_dimension='TOTAL'):
    """Acumula un resultado de KPI para guardarlo en kpi_resultado al final de la sección."""
    resultados_kpi.append({'ejecucion_id': EJECUCION_ID, 'kpi_id': kpi_id, 'dimension': dimension,
                           'valor_dimension': str(valor_dimension), 'metrica': metrica,
                           'valor': None if pd.isna(valor) else round(float(valor), 4)})

def leer(sql):
    """Consulta el DWH en MySQL y devuelve un DataFrame."""
    return pd.read_sql(text(sql), engine)

print(f"Catálogo: {len(KPI_DEFINICIONES)} KPIs ({len(nuevos)} nuevos) | Ejecución: {EJECUCION_ID}")
```

Catálogo: 7 KPIs (0 nuevos) | Ejecución: 682cea44-1d16-4c36-ae5b-2fe2ae573ab7

6.1 KPI_DISTRIBUCION_PRODUCTOS

Distribución de clientes por nº de productos contratados. Permite identificar cuántos clientes son **mono-producto** (riesgo de fuga) frente a los muy vinculados.

- `numero_productos` = suma de los 45 indicadores `ind_*` por cliente (`total_productos_activos` en el DWH).
- Para cada valor: `total_clientes` , `% clientes` , `patrimonio_medio` y `margen_medio` .
- KPI global: **% de clientes mono-producto** y **% de clientes sin productos**.

Resultado (snapshot 11/09/2026):

- El **27,0 %** de los clientes es **mono-producto** y el **20,05 %** no tiene ningún producto activo: casi la mitad de la cartera tiene una vinculación débil.
- El valor del cliente crece con la vinculación: el patrimonio medio pasa de 5.142 € (0 productos) a 50.722 € (4 productos) y supera los 200.000 € a partir de 8 productos. El margen medio pasa de 53 € a 533 €.

In [155...

```
kpi_dist = leer("""
SELECT numero_productos, total_clientes, pct_clientes, patrimonio_medio, margen_medio
FROM agg_distribucion_productos
WHERE tiempo_id = {TIEMPO_ID_SNAPSHOT}
ORDER BY numero_productos""")

pct_mono = kpi_dist.loc[kpi_dist['numero_productos'] == 1, 'pct_clientes'].sum()
pct_sin = kpi_dist.loc[kpi_dist['numero_productos'] == 0, 'pct_clientes'].sum()
registrar('KPI_DISTRIBUCION_PRODUCTOS', 'pct_clientes_mono_producto', pct_mono)
registrar('KPI_DISTRIBUCION_PRODUCTOS', 'pct_clientes_sin_productos', pct_sin)
for fila in kpi_dist.itertuples():
    for metrica in ['total_clientes', 'pct_clientes', 'patrimonio_medio', 'margen_medio']:
        registrar('KPI_DISTRIBUCION_PRODUCTOS', metrica, getattr(fila, metrica), 'numero_productos', fila.numero_productos)

print(f"% clientes mono-producto: {pct_mono:.1f}% | % clientes sin productos: {pct_sin:.1f}%\n")
kpi_dist
```

% clientes mono-producto: 27.0% | % clientes sin productos: 20.1%

Out[155...

	numero_productos	total_clientes	pct_clientes	patrimonio_medio	margen_medio
0	0	1969	20.05	5141.81	52.80
1	1	2653	27.01	8026.62	67.23
2	2	1552	15.80	4725.28	108.91
3	3	1355	13.80	28556.21	190.25
4	4	1020	10.38	50722.10	155.82
5	5	688	7.00	51563.71	177.38
6	6	327	3.33	71101.42	401.93
7	7	161	1.64	117074.02	461.34
8	8	60	0.61	217806.75	533.35
9	9	23	0.23	278030.35	738.12
10	10	12	0.12	301837.01	720.63
11	11	2	0.02	616645.87	988.38

6.2 KPI_ENDEUDAMIENTO

Mide el apalancamiento de cada cliente respecto a su patrimonio:

ratio_endeudamiento = (credito_utilizado + revolving_utilizado + total_hipoteca) / total_patrimonio

El ratio por cliente se guarda en `fact_cliente_snapshot` (junto con `deuda_total` y `nivel_riesgo_endeudamiento`), así que la lista de clientes a vigilar se obtiene con una consulta. Como KPI se registra el **% de clientes en riesgo crítico (ratio > 1)**, en total y por segmento, y la distribución por nivel de riesgo. Los umbrales (0,3 / 0,6 / 1) son orientativos y el equipo puede ajustarlos.

Resultado (snapshot 11/09/2026):

- 1.425 clientes (14,5 %)** están en riesgo **crítico**: su deuda supera su patrimonio en el banco. Otros 347 clientes no tienen patrimonio y no se puede calcular su ratio.
- Por perfil: Pensionado 16,7 %, Conservador 8,3 %, Juvenil 0 %.
- El ratio mediano entre los clientes con deuda es 17,3, porque casi toda la deuda es hipotecaria.

Limitación: **totalpasta** es el patrimonio gestionado por el banco, no la riqueza total del cliente, por lo que el ratio tiende a sobrestimar el endeudamiento real.

In [156...

```
riesgo = leer(f"""
    SELECT s.tipo_inversion, f.nivel_riesgo_endeudamiento, COUNT(*) AS clientes
    FROM fact_cliente_snapshot f
    JOIN dim_cliente c ON c.cliente_sk = f.cliente_sk
    JOIN dim_segmento s ON s.segmento_sk = c.segmento_sk
    WHERE f.tiempo_id = {TIEMPO_ID_SNAPSHOT}
    GROUP BY s.tipo_inversion, f.nivel_riesgo_endeudamiento""")

orden_riesgo = ['Bajo', 'Moderado', 'Alto', 'Crítico', 'Sin patrimonio']
tabla_riesgo = (riesgo.pivot_table(index='tipo_inversion', columns='nivel_riesgo_endeudamiento',
                                   values='clientes', aggfunc='sum', fill_value=0)
               .reindex(columns=orden_riesgo, fill_value=0))
tabla_riesgo.loc['TOTAL'] = tabla_riesgo.sum()
tabla_riesgo['% crítico'] = (tabla_riesgo['Crítico'] / tabla_riesgo[orden_riesgo].sum(axis=1) * 100).round(1)

registrar('KPI_ENDEUDAMIENTO', 'pct_clientes_riesgo_critico', tabla_riesgo.loc['TOTAL', '% crítico'])
for nivel in orden_riesgo:
    registrar('KPI_ENDEUDAMIENTO', 'total_clientes', tabla_riesgo.loc['TOTAL', nivel], 'nivel_riesgo', nivel)
for segmento, fila in tabla_riesgo.drop(index='TOTAL').iterrows():
    registrar('KPI_ENDEUDAMIENTO', 'pct_clientes_riesgo_critico', fila['% crítico'], 'tipo_inversion', segmento)

mediana = leer(f"SELECT ratio_endeudamiento FROM fact_cliente_snapshot "
               f"WHERE tiempo_id = {TIEMPO_ID_SNAPSHOT} AND deuda_total > 0 AND ratio_endeudamiento IS NOT NULL")
registrar('KPI_ENDEUDAMIENTO', 'ratio_mediano_clientes_con_deuda', mediana['ratio_endeudamiento'].median())

print(f"% clientes en riesgo crítico (ratio > 1): {tabla_riesgo.loc['TOTAL', '% crítico']}%")
print(f"Ratio mediano entre los clientes con deuda: {mediana['ratio_endeudamiento'].median():.2f}\n")
tabla_riesgo
```

% clientes en riesgo crítico (ratio > 1): 14.5%
Ratio mediano entre los clientes con deuda: 17.26

Out[156...

nivel_riesgo_endeudamiento	Bajo	Moderado	Alto	Crítico	Sin patrimonio	% crítico
tipo_inversion						
Inversión Perfil Conservador	871	3	6	84	50	8.3
Inversión Perfil Juvenil	771	0	0	0	3	0.0
Inversión Perfil Pensionado	6282	63	54	1341	294	16.7
TOTAL	7924	66	60	1425	347	14.5

6.3 KPI_POR_NIVEL_RENTA

Comportamiento de la cartera según el nivel socioeconómico (**nivren** decodificado a Bajo ... Alto), cruzado con patrimonio, rentabilidad, vinculación e inversión. Sirve para priorizar en qué segmento de renta enfocar la actividad comercial: por volumen, por rentabilidad por cliente o por oportunidad de venta cruzada.

Resultado (snapshot 11/09/2026):

- **Volumen:** los niveles Bajo y Medio-Bajo reúnen el 72,9 % de los clientes.
- **Rentabilidad:** el nivel Medio-Alto (5,6 % de los clientes) tiene un margen medio de 321 €, 3,5 productos por cliente y un 62,0 % con inversión.
- **Oportunidad de venta cruzada:** el nivel Alto tiene el mayor margen medio (563 €), pero menos productos (2,7) y menos inversión (41,5 %) que el Medio-Alto. Tiene recorrido para productos de inversión.

In [157...

```
kpi_renta = leer(f"""
    SELECT nivel_renta, total_clientes, pct_clientes, patrimonio_medio, margen_medio,
           productos_medios, pct_con_inversion
    FROM agg_nivel_renta
    WHERE tiempo_id = {TIEMPO_ID_SNAPSHOT}
    ORDER BY nivren_codigo""")

for fila in kpi_renta.itertuples():
    for metrica in ['total_clientes', 'pct_clientes', 'patrimonio_medio', 'margen_medio',
                   'productos_medios', 'pct_con_inversion']:
        registrar('KPI_POR_NIVEL_RENTA', metrica, getattr(fila, metrica), 'nivel_renta', fila.nivel_renta)
kpi_renta
```

Out[157...

	nivel_renta	total_clientes	pct_clientes	patrimonio_medio	margen_medio	productos_medios	pct_con_inversion
0	Sin datos	5	0.05	5636.57	27.10	0.80	20.00
1	Bajo	3996	40.68	12226.41	73.84	1.76	22.82
2	Medio-Bajo	3161	32.18	20754.22	134.68	2.13	35.68
3	Medio	2067	21.04	25408.60	161.56	2.63	41.65
4	Medio-Alto	552	5.62	101152.48	321.49	3.53	61.96
5	Alto	41	0.42	200762.69	562.74	2.73	41.46

6.4 KPI_POR_SEGMENTO

KPI central para comparar los 3 perfiles de inversión (Juvenil, Conservador, Pensionado) en todas las dimensiones de gestión comercial. Es la base para diseñar los 3 productos de inversión del objetivo 3:

- **Juvenil:** inversión para estudios
- **Conservador:** inversión para la primera vivienda
- **Pensionado:** inversión para la jubilación

Resultado (snapshot 11/09/2026):

- **Pensionado:** 81,8 % de los clientes y **95,5 % del margen comercial**. Es el perfil a retener; aun así, 5.135 clientes todavía no invierten (mercado del producto de jubilación).
- **Conservador:** 10,3 % de los clientes, 4,0 % del margen y el porcentaje de inversión más bajo (16,0 %). Hay 852 clientes sin inversión para el producto de primera vivienda.
- **Juvenil:** 7,9 % de los clientes y solo el 0,4 % del margen (7 € de margen medio), pero con 2,6 productos por cliente, más vinculado que el Conservador. Hay 574 clientes sin inversión para el producto de estudios.

In [158...

```
kpi_seg = leer(f"""
    SELECT s.tipo_inversion, a.total_clientes, a.pct_clientes, a.patrimonio_medio, a.margen_medio,
           a.pct_margen, a.productos_medios, a.clientes_con_inversion, a.pct_con_inversion,
           a.clientes_riesgo_critico
    FROM agg_segmento a
    JOIN dim_segmento s ON s.segmento_sk = a.segmento_sk
    WHERE a.tiempo_id = {TIEMPO_ID_SNAPSHOT}
    ORDER BY s.orden_perfil""")

for fila in kpi_seg.itertuples():
    for metrica in ['total_clientes', 'pct_clientes', 'patrimonio_medio', 'margen_medio', 'pct_margen',
                   'productos_medios', 'pct_con_inversion']:
        registrar('KPI_POR_SEGMENTO', metrica, getattr(fila, metrica), 'tipo_inversion', fila.tipo_inversion)
# Mercado potencial del producto de inversión de cada perfil: clientes que aún no invierten
registrar('KPI_POR_SEGMENTO', 'clientes_sin_inversion', fila.total_clientes - fila.clientes_con_inversion,
          'tipo_inversion', fila.tipo_inversion)

kpi_seg
```

Out[158...

	tipo_inversion	total_clientes	pct_clientes	patrimonio_medio	margen_medio	pct_margen	productos_medios	clientes_con_inversion	pct_con_inversion	clientes_riesgo_critico
0	Inversión Perfil Juvenil	774	7.88	2419.59	7.06	0.44	2.64	200	25.84	0
1	Inversión Perfil Conservador	1014	10.32	4442.20	49.88	4.03	1.76	162	15.98	84
2	Inversión Perfil Pensionado	8034	81.80	27968.49	149.29	95.54	2.17	2899	36.08	1341

6.5 KPI_PENETRACION_PRODUCTO

Grado de adopción de cada uno de los 45 productos sobre el total de clientes ($\text{clientes_con_producto} / \text{total_clientes} \times 100$), a partir de `fact_producto_cliente`. También se agrega por categoría (Cuentas, Tarjetas, Inversion_Ahorro, Financiacion, Otros), contando a cada cliente una sola vez por categoría. Permite detectar productos masificados (producto ancla) y productos con poca adopción, candidatos a venta cruzada.

Resultado (snapshot 11/09/2026):

- Por categoría: Cuentas 56,7 %, Inversión/Ahorro 33,2 %, Tarjetas 18,4 %, Financiación 14,7 %.
- El producto ancla es la **Cuenta Expansión** (34,9 %).
- **9 de los 45 productos no tienen ningún cliente** (por ejemplo, seguros de vida-ahorro colectivos y rentas temporales). Conviene tratarlos como productos inactivos en el análisis.

In [159...

```
kpi_pen = leer(f"""
    SELECT p.categoria, p.nombre_producto, p.codigo_indicador, a.clientes_con_producto, a.pct_penetracion
    FROM agg_producto a
    JOIN dim_producto p ON p.producto_sk = a.producto_sk
    WHERE a.tiempo_id = {TIEMPO_ID_SNAPSHOT}
    ORDER BY a.pct_penetracion DESC""")

# Penetración por categoría: clientes distintos con al menos un producto de la categoría
kpi_pen_cat = leer(f"""
    SELECT p.categoria, COUNT(DISTINCT f.cliente_sk) AS clientes_con_categoria
    FROM fact_producto_cliente f
    JOIN dim_producto p ON p.producto_sk = f.producto_sk
    WHERE f.tiempo_id = {TIEMPO_ID_SNAPSHOT}
    GROUP BY p.categoria""")

total = len(fact_cliente_snapshot)
kpi_pen_cat['pct_penetracion'] = (kpi_pen_cat['clientes_con_categoria'] / total * 100).round(2)
kpi_pen_cat = kpi_pen_cat.sort_values('pct_penetracion', ascending=False)

for fila in kpi_pen.itertuples():
    registrar('KPI_PENETRACION_PRODUCTO', 'pct_penetracion', fila.pct_penetracion, 'producto', fila.codigo_indicador)
for fila in kpi_pen_cat.itertuples():
    registrar('KPI_PENETRACION_PRODUCTO', 'pct_penetracion', fila.pct_penetracion, 'categoria', fila.categoria)
sin_clientes = (kpi_pen['clientes_con_producto'] == 0).sum()
registrar('KPI_PENETRACION_PRODUCTO', 'productos_sin_clientes', sin_clientes)

print("Penetración por categoría:")
print(kpi_pen_cat.to_string(index=False))
print(f"\nProductos sin ningún cliente: {sin_clientes} de {len(kpi_pen)}")
print("\nTop 10 productos:")
kpi_pen.head(10)
```

Penetración por categoría:

	categoria	clientes_con_categoria	pct_penetracion
	Cuentas	5566	56.67
Inversion_Ahorro		3261	33.20
	Tarjetas	1803	18.36
	Financiacion	1439	14.65

Productos sin ningún cliente: 9 de 45

Top 10 productos:

Out[159...

	categoria	nombre_producto	codigo_indicador	clientes_con_producto	pct_penetracion
0	Cuentas	Cuenta Expansión	ind_cuentaexpansion	3427	34.891
1	Inversion_Ahorro	Estalvi Inversión	ind_estalinv	3261	33.201
2	Inversion_Ahorro	Estalvi (Ahorro)	ind_estalvi	2994	30.483
3	Inversion_Ahorro	Depósitos	ind_diposit	1354	13.785
4	Financiacion	Hipoteca	ind_hipoteca	1322	13.460
5	Financiacion	Hipoteca tipo variable	ind_hipvar	1116	11.362
6	Inversion_Ahorro	CIALP (Cuenta Individual Ahorro L/P)	ind_cialp	951	9.682
7	Inversion_Ahorro	Renta fija	ind_rfival	817	8.318
8	Cuentas	Cuenta Experiencia	ind_cuentaexperiencia	800	8.145
9	Cuentas	Cuenta Proyección	ind_cuentaproyeccion	657	6.689

6.6 KPI_POR_PROVINCIA

Segmenta y compara la cartera por provincia de residencia (código INE real del cliente; 99 = extranjero): nº de clientes, patrimonio total y medio, y margen medio. Permite ver dónde se concentra el negocio y qué provincias tienen clientes de mayor valor medio.

Resultado (snapshot 11/09/2026):

- Barcelona (23,5 %), Alicante (15,1 %) y los residentes en el extranjero (8,1 %) reúnen el **46,7 %** de los clientes.
- Por valor medio destacan Asturias (39.115 € de patrimonio medio, frente a 30.105 € en Barcelona) y, entre las provincias con más clientes, Girona (242 € de margen medio).

In [160...

```
kpi_prov = leer(f"""
SELECT g.provincia_nombre, g.comunidad_autonoma, g.es_extranjero, a.total_clientes, a.pct_clientes,
       a.patrimonio_total, a.patrimonio_medio, a.margen_medio
FROM   agg_provincia a
JOIN   dim_geografia g ON g.geografia_sk = a.geografia_sk
WHERE  a.tiempo_id = {TIEMPO_ID_SNAPSHOT}
ORDER BY a.total_clientes DESC""")

for fila in kpi_prov.itertuples():
    for metrica in ['total_clientes', 'pct_clientes', 'patrimonio_total', 'patrimonio_medio', 'margen_medio']:
        registrar('KPI_POR_PROVINCIA', metrica, getattr(fila, metrica), 'provincia', fila.provincia_nombre)
top3 = kpi_prov.head(3)['pct_clientes'].sum()
registrar('KPI_POR_PROVINCIA', 'pct_clientes_top3_provincias', top3)

print(f"Provincias con clientes: {len(kpi_prov)} | % de clientes en las 3 primeras: {top3:.1f}%\n")
kpi_prov.head(15)
```

Provincias con clientes: 53 | % de clientes en las 3 primeras: 46.7%

Out[160...

	provincia_nombre	comunidad_autonoma	es_extranjero	total_clientes	pct_clientes	patrimonio_total	patrimonio_medio	margen_medio
0	Barcelona	Cataluña	0	2309	23.51	69513577.51	30105.49	133.31
1	Alicante/Alacant	Comunitat Valenciana	0	1487	15.14	30203413.78	20311.64	103.66
2	Extranjero	Extranjero	1	792	8.06	10030113.46	12664.28	147.74
3	Murcia	Región de Murcia	0	746	7.60	14405566.70	19310.41	106.02
4	Madrid	Comunidad de Madrid	0	724	7.37	17907274.60	24733.80	159.17
5	Valencia/València	Comunitat Valenciana	0	570	5.80	10153927.21	17813.91	120.08
6	Asturias	Principado de Asturias	0	535	5.45	20926761.93	39115.44	138.39
7	Girona	Cataluña	0	274	2.79	6198543.93	22622.42	241.74
8	Tarragona	Cataluña	0	221	2.25	3520953.33	15931.92	137.24
9	Málaga	Andalucía	0	196	2.00	3589646.65	18314.52	141.80
10	Illes Balears	Illes Balears	0	177	1.80	3898292.24	22024.25	107.57
11	Lleida	Cataluña	0	149	1.52	2277787.19	15287.16	120.14
12	A Coruña	Galicia	0	139	1.42	3125804.32	22487.80	88.82
13	Castellón/Castelló	Comunitat Valenciana	0	137	1.39	2576030.41	18803.14	120.24
14	León	Castilla y León	0	118	1.20	3564106.80	30204.29	94.19

6.7 KPI_TOP_CLIENTES_PATRIMONIO

Identifica a los 100 clientes con más patrimonio de la cartera, con su segmento, renta y provincia, para que los gestores comerciales o de banca privada los revisen. Sirve para:

- Priorizar la atención comercial sobre la cartera de mayor valor.
- Detectar **fuga de margen**: clientes con mucho patrimonio y margen comercial bajo o negativo.
- Tener una lista de trabajo para campañas de fidelización o para revisar el riesgo concentrado en pocos clientes.

KPI global: % del patrimonio total que concentran los 100 primeros clientes.

Resultado (snapshot 11/09/2026):

- Los 100 primeros clientes (1 % de la cartera) concentran el **24,1 % del patrimonio**: un riesgo de concentración relevante.
- Todos son del perfil Pensionado.
- Solo 3 tienen un margen comercial inferior a la mediana de la cartera (35,90 €): son los candidatos a revisar por fuga de margen.

In [161...

```
top_clientes = leer(f"""
SELECT t.ranking, c.id_cliente, s.tipo_inversion, c.nivel_renta, g.provincia_nombre,
       t.total_patrimonio, t.pct_patrimonio_cartera, t.margen_comercial, t.total_productos_activos
FROM   agg_top_clientes_patrimonio t
JOIN   dim_cliente c   ON c.cliente_sk = t.cliente_sk
JOIN   dim_segmento s  ON s.segmento_sk = c.segmento_sk
JOIN   dim_geografia g ON g.geografia_sk = c.geografia_sk
WHERE  t.tiempo_id = {TIEMPO_ID_SNAPSHOT}
ORDER BY t.ranking""")

pct_top100 = top_clientes['pct_patrimonio_cartera'].sum()
margen_mediano = fact_cliente_snapshot['margen_comercial'].median()
fuga_margen = (top_clientes['margen_comercial'] < margen_mediano).sum()
registrar('KPI_TOP_CLIENTES_PATRIMONIO', 'pct_patrimonio_top100', pct_top100)
registrar('KPI_TOP_CLIENTES_PATRIMONIO', 'clientes_top100_margen_bajo_mediana', fuga_margen)

print(f"Los 100 primeros clientes concentran el {pct_top100:.1f}% del patrimonio de la cartera")
print(f"Clientes del Top 100 con margen inferior a la mediana de la cartera ({margen_mediano:.2f} €): {fuga_margen}\n")
top_clientes.head(15)
```

Los 100 primeros clientes concentran el 24.1% del patrimonio de la cartera
Clientes del Top 100 con margen inferior a la mediana de la cartera (35.90 €): 3

Out[161...

	ranking	id_cliente	tipo_inversion	nivel_renta	provincia_nombre	total_patrimonio	pct_patrimonio_cartera	margen_comercial	total_productos_activos
0	1	711	Inversión Perfil Pensionado	Alto	Barcelona	4798534.49	2.077	10434.39	3
1	2	116	Inversión Perfil Pensionado	Medio-Alto	Asturias	2978202.22	1.289	5405.60	8
2	3	846	Inversión Perfil Pensionado	Medio-Alto	Barcelona	1808010.92	0.782	3492.44	9
3	4	216	Inversión Perfil Pensionado	Medio-Bajo	Valencia/València	1245550.52	0.539	2122.51	3
4	5	2213	Inversión Perfil Pensionado	Medio-Alto	Madrid	1157299.42	0.501	1725.92	11
5	6	438	Inversión Perfil Pensionado	Alto	Barcelona	1146056.67	0.496	1962.81	10
6	7	36	Inversión Perfil Pensionado	Medio-Alto	Asturias	1081485.57	0.468	2413.57	5
7	8	515	Inversión Perfil Pensionado	Bajo	Barcelona	1014844.69	0.439	3704.11	7
8	9	4016	Inversión Perfil Pensionado	Medio-Alto	Alicante/Alacant	992799.80	0.430	1856.87	4
9	10	3038	Inversión Perfil Pensionado	Medio-Alto	Madrid	952979.06	0.412	2331.09	6
10	11	888	Inversión Perfil Pensionado	Medio-Alto	Barcelona	896137.71	0.388	1801.92	9
11	12	1649	Inversión Perfil Pensionado	Medio-Alto	Illes Balears	825151.65	0.357	120.95	6
12	13	426	Inversión Perfil Pensionado	Medio-Alto	Extranjero	778952.75	0.337	960.34	5
13	14	8486	Inversión Perfil Pensionado	Bajo	Extranjero	771906.95	0.334	3089.84	1
14	15	799	Inversión Perfil Pensionado	Medio-Alto	Madrid	764463.63	0.331	1967.46	8

6.8 Guardado de los resultados de la ejecución

Se insertan en `kpi_resultado` todos los valores calculados en esta ejecución y se consulta el histórico de los KPIs globales (`dimension = 'TOTAL'`).

In [162...

```
df_resultados = pd.DataFrame(resultados_kpi)
df_resultados.to_sql('kpi_resultado', engine, if_exists='append', index=False, chunksize=1000, method='multi')
print(f"✅ {len(df_resultados)} resultados guardados en kpi_resultado (ejecución {EJECUCION_ID})\n")

historico = leer("""
    SELECT e.fecha_ejecucion, d.nombre AS kpi, r.metrica, r.valor
    FROM kpi_resultado r
    JOIN kpi_ejecucion e ON e.ejecucion_id = r.ejecucion_id
    JOIN kpi_definicion d ON d.kpi_id = r.kpi_id
    WHERE r.dimension = 'TOTAL'
    ORDER BY e.fecha_ejecucion DESC, d.kpi_id, r.metrica""")
historico
```

✅ 438 resultados guardados en kpi_resultado (ejecución 682cea44-1d16-4c36-ae5b-2fe2ae573ab7)

Out[162...

	fecha_ejecucion	kpi	metrica	valor
0	2026-09-30 04:54:21	Distribución de clientes por nº de productos	pct_clientes_mono_producto	27.0100
1	2026-09-30 04:54:21	Distribución de clientes por nº de productos	pct_clientes_sin_productos	20.0500
2	2026-09-30 04:54:21	Ratio de endeudamiento	pct_clientes_riesgo_critico	14.5000
3	2026-09-30 04:54:21	Ratio de endeudamiento	ratio_mediano_clientes_con_deuda	17.2642
4	2026-09-30 04:54:21	Penetración de producto	productos_sin_clientes	9.0000
5	2026-09-30 04:54:21	Cartera por provincia	pct_clientes_top3_provincias	46.7100
6	2026-09-30 04:54:21	Concentración de patrimonio en el Top 100	clientes_top100_margen_bajo_mediana	3.0000
7	2026-09-30 04:54:21	Concentración de patrimonio en el Top 100	pct_patrimonio_top100	24.1170
8	2026-09-29 20:05:29	Distribución de clientes por nº de productos	pct_clientes_mono_producto	27.0100
9	2026-09-29 20:05:29	Distribución de clientes por nº de productos	pct_clientes_sin_productos	20.0500
10	2026-09-29 20:05:29	Ratio de endeudamiento	pct_clientes_riesgo_critico	14.5000
11	2026-09-29 20:05:29	Ratio de endeudamiento	ratio_mediano_clientes_con_deuda	17.2642
12	2026-09-29 20:05:29	Penetración de producto	productos_sin_clientes	9.0000
13	2026-09-29 20:05:29	Cartera por provincia	pct_clientes_top3_provincias	46.7100
14	2026-09-29 20:05:29	Concentración de patrimonio en el Top 100	clientes_top100_margen_bajo_mediana	3.0000
15	2026-09-29 20:05:29	Concentración de patrimonio en el Top 100	pct_patrimonio_top100	24.1170
16	2026-09-29 18:57:01	Distribución de clientes por nº de productos	pct_clientes_mono_producto	27.0100
17	2026-09-29 18:57:01	Distribución de clientes por nº de productos	pct_clientes_sin_productos	20.0500
18	2026-09-29 18:57:01	Ratio de endeudamiento	pct_clientes_riesgo_critico	14.5000
19	2026-09-29 18:57:01	Ratio de endeudamiento	ratio_mediano_clientes_con_deuda	17.2642
20	2026-09-29 18:57:01	Penetración de producto	productos_sin_clientes	9.0000
21	2026-09-29 18:57:01	Cartera por provincia	pct_clientes_top3_provincias	46.7100
22	2026-09-29 18:57:01	Concentración de patrimonio en el Top 100	clientes_top100_margen_bajo_mediana	3.0000
23	2026-09-29 18:57:01	Concentración de patrimonio en el Top 100	pct_patrimonio_top100	24.1170

7. VALIDACIÓN DEL PROCESO ETL

Comprobaciones finales sobre MySQL: nº de filas por tabla, integridad referencial (registros huérfanos) y cuadre de importes entre staging y el DWH.

In [163...

```
conteos = pd.concat([leer(f"SELECT '{t}' AS tabla, COUNT(*) AS filas FROM {t}")
                    for t in [TABLA_STAGING] + TABLAS_DWH], ignore_index=True)
print(conteos.to_string(index=False))

huerfanos = leer("""
    SELECT 'fact_cliente_snapshot sin dim_cliente' AS control, COUNT(*) AS filas
    FROM fact_cliente_snapshot f LEFT JOIN dim_cliente c ON c.cliente_sk = f.cliente_sk WHERE c.cliente_sk IS NULL
```

```
UNION ALL
SELECT 'fact_producto_cliente sin dim_producto', COUNT(*)
  FROM fact_producto_cliente f LEFT JOIN dim_producto p ON p.producto_sk = f.producto_sk WHERE p.producto_sk IS NULL
UNION ALL
SELECT 'dim_cliente sin dim_geografia', COUNT(*)
  FROM dim_cliente c LEFT JOIN dim_geografia g ON g.geografia_sk = c.geografia_sk WHERE g.geografia_sk IS NULL
UNION ALL
SELECT 'kpi_resultado sin kpi_definicion', COUNT(*)
  FROM kpi_resultado r LEFT JOIN kpi_definicion d ON d.kpi_id = r.kpi_id WHERE d.kpi_id IS NULL""")
print("\n" + huerfanos.to_string(index=False))

cuadre = leer(f"""
SELECT (SELECT ROUND(SUM(totalpasta), 2) FROM {TABLA_STAGING}) AS patrimonio_staging,
       (SELECT ROUND(SUM(total_patrimonio), 2) FROM fact_cliente_snapshot
        WHERE tiempo_id = {TIEMPO_ID_SNAPSHOT}) AS patrimonio_dwh""")
print("\n" + cuadre.to_string(index=False))

assert huerfanos['filas'].sum() == 0, "Hay registros huérfanos"
assert abs(cuadre.iloc[0, 0] - cuadre.iloc[0, 1]) < 1, "El patrimonio no cuadra entre staging y el DWH"
print("\n✅ ETL validado: sin huérfanos y con importes cuadrados")
```

	tabla	filas
	stg_clientes_original	9822
	dim_segmento	3
	dim_geografia	53
	dim_producto	45
	dim_tiempo	495
	dim_cliente	9822
	fact_cliente_snapshot	9822
	fact_producto_cliente	21260
	agg_segmento	3
	agg_nivel_renta	6
	agg_provincia	53
	agg_producto	45
	agg_distribucion_productos	12
	agg_top_clientes_patrimonio	100
	kpi_definicion	7
	kpi_ejecucion	3
	kpi_resultado	1314

	control	filas
	fact_cliente_snapshot sin dim_cliente	0
	fact_producto_cliente sin dim_producto	0
	dim_cliente sin dim_geografia	0
	kpi_resultado sin kpi_definicion	0

patrimonio_staging	patrimonio_dwh
231076038.57	231076038.57

✅ ETL validado: sin huérfanos y con importes cuadrados